CHRONOS System Specification v0.2

Oct 7, 2026 · @Arun

About this document

This specification fixes what CHRONOS is, how it works, in what order we build it, and what we deliberately leave out. Code must follow it; it is the contract for the first implementation.

Version: 0.2 (decisions D-01 to D-26 reviewed and approved; open questions Q1 and Q6 remain) · **Team:** Arun B, Viswajith G S · **Status:** architecture and specification stage; no implementation yet.

Change log: v0.1 to v0.2

v0.2 folds in the decision review: six amended decisions, one re-worded decision, four new decisions (D-23 to D-26), a reordered phase plan that reaches the core idea first, and the corrected estimate of 35.5 joint weeks (v0.1 said 37). Questions Q1 and Q6 are still **OPEN**.

Change	What changed	Sections
D-03 (re-worded)	Simulation mode only in v1. A live-network recorder is a possible post-v1 idea and must never enter the replay core	9, B
D-04 (amended)	Node state has a volatile zone (lost on crash) and a durable zone (survives a crash, stands for disk). Each node declares a state codec through the Node API. Durable changes commit at step end, atomically with the step's events. The durable zone and restart arrive in Phase 12b	9, 10, 14, 18, 21
D-07 (amended)	Vectors have N + 1 slots (nodes in declared order, `env` last) and every table, diagram, golden file and stored event shows all of them. Example values recomputed	10, 11, 28

Change	What changed	Sections
D-13 (amended)	Fault rules are keyed by time or by identity, never by a global counter. A wildcard sender is allowed only if flagged nonLocal, which voids law M8 for that run	14, 18, 19, 25
D-16 (amended)	The first causal divergence is the earliest causal root. Taint is computed separately inside each timeline; roots from both timelines are ordered by virtual time, node order and event id. The explanation path uses an anchor rule, validated in Phase 14a	20, 21, 25
D-17 (amended)	The engine records the last writer of every state field. Invariants read state through a recording accessor, and a violation gets a WITNESS edge to the last writer of each field it read	9, 10, 12, 14, 21
D-18 (amended)	A static, self-contained HTML report generated by the CLI is required (Phase 15a). A live local viewer is Stretch (post-v1)	4, 5, 7, 13, 16, 21, 23, 24
D-23 (new)	OMISSION edge kind: a fault event is linked to the timer firing that noticed the silence it caused	10, 12, 18
D-24 (new)	Events on `env` are keyed by what they concern (`drop:<messageId>` , `fault:<ruleId>#<n>` , `iv:<interventionId>` , `inv:<name>#<n>`). The `env` counter feeds clocks only	10, 19, 20
D-25 (new)	An INTERVENTION_APPLIED event on `env` marks each intervention at the branch point. `origin` becomes part of the content digest, so law M10 is well defined	10, 17, 19, 20, 25

Change	What changed	Sections
D-26 (new)	The Node API moves into its own module, `chronos-sdk` (eight modules). Dependency corrections: LogReader and EventSink live in the model; replay and branch decision sources live in analysis; the store owns catalogue persistence and segment resolution; the timeline manager owns timeline id policy and branch creation; the CLI depends on analysis, runtime, store, demo, sdk and model	21, 23, A
Phase plan	Phase numbers stay as identifiers. Suffix a marks the thin core of a phase, b the deeper follow-up. The execution order differs from the numeric order and reaches the core idea (record, explain, branch, intervene, compare) first. Milestones are now A, B, C, S, D, E	0, 7, 24, 26, 28
Estimate	37 changed to 35.5 joint weeks (Phase 15 shrinks from 3.5 to 2 weeks because the web viewer became a static report). Milestone S is about 24 joint weeks	24, 27
Clarifications	Every edge kind carries clocks like a message, and `env` follows the same Lamport and vector rules. Invariant field reads are zone-qualified. An unwritten field's last writer is the node's start event, and after a restart it is the restart event. The stored state format reserves a zone tag from Phase 8. The default `why` follows certain and OMISSION edges. Cut-line order updated	10, 11, 12, 14, 16, 24
Testing and risk	New laws M13 (environment identity) and M14 (witnesses); new risk R18 (explanation-path rules not yet validated)	25, 27
Open questions	Q8 resolved; Q5 resolved by D-03; Q2, Q3, Q4, Q7, Q9 and Q10 confirmed; Q1 text updated for the new arithmetic only; Q6 untouched	B

Still open: Q1 (deadline and weekly hours) and Q6 (evaluation requirements of your institution). Neither has been answered or assumed.

How to read it

- Sections 1–5 explain the problem and the boundaries of the project. Read these first, together.
- Sections 6–20 explain the ideas and the models. Every hard idea is taught in four steps: **Plain idea**, **Tiny example**, **Technical view**, **In CHRONOS**.
- Sections 21–23 name the components and show how they connect.
- Sections 24–28 are the plan: phases, tests, benchmarks, risks, and what to build first.
- Appendix A is the architecture diagram in text form. Appendix B is the decision log: every architectural choice with its alternatives.

Conventions

- IDs: **FR-xx** functional requirement, **NFR-xx** non-functional requirement, **D-xx** architectural decision, **C1–C18** component, **AD-x** acceptance demonstration. Phases keep the numbers 1–17 of the project plan as identifiers; a suffix a marks the thin core of a phase and b its deeper follow-up, and the execution order (Section 24) differs from the numeric order.
- Priorities use MoSCoW: **Must**, **Should**, **Could**. **Stretch** means: only after every Must item has been demonstrated.
- Where this document says *recommend*, the choice is recorded in Appendix B and stays open until both of us confirm it. After confirmation, a decision changes only by adding a new decision that supersedes it, with the consequences written down.
- Two running examples appear throughout. **Ping-pong** (nodes A and B) is the tiny example for clocks and events. **RE** (a small replicated key-value store with a primary, a backup and a client) is the flagship example, introduced in Section 1.

1. Problem statement

When a distributed system fails, a developer can see the symptom but usually cannot replay the run, rewind it, or ask what would have happened if one thing had gone differently. CHRONOS exists to close that gap.

The running example (RE)

RE is a tiny replicated key-value store with three nodes: `client`, `p` (primary) and `b` (backup).

1. The client sends writes to `p`. `p` stores each write and copies it to `b`.
2. `p` sends a heartbeat to `b` every 100 ms.
3. If `b` hears no heartbeat for 300 ms, it assumes `p` is dead and promotes itself to primary.
4. A network partition cuts the link between `p` and `b` for 400 ms. `p` is alive, but `b` cannot hear it.
5. `b` promotes itself. Now **two primaries exist** (split brain). The client writes to `p`, then switches to `b`. After the partition heals, the two sides disagree and a write is lost.

The developer's only symptom: *write W7 is missing*. The real cause happened seconds earlier, on a different machine, in a different order than any log suggests.

What the developer actually needs

1. **Reproduce** the failing run exactly, every time.
2. **Navigate** it: see the state of every node, and every message in flight, at any moment.
3. **Explain** it: find the earliest event that put the system on the road to failure, and the chain from there to the symptom.
4. **Experiment**: change one thing in the past (heal the partition earlier, drop a message, crash a node) and see whether the failure still happens.
5. **Compare**: put two executions side by side and find where they first parted ways.

Why existing tools fall short

Tool family	Good at	Cannot do for us
Log files	Showing what each machine chose to print	Order events across machines; rewind; replay
Single-process debugger	Stepping through one program	See several nodes at once; handle message ordering; run backward
Distributed tracing	Following one request across services	Reconstruct full state; replay; change the past
Chaos testing tools	Injecting failures into a running system	Reproduce the exact failing run; explain the cause
Model checkers	Exploring many interleavings of a formal model	Debug the real Java code of a running application

CHRONOS is the combination none of these provides: **record, rewind, replay, branch, and compare executions of a distributed Java application, with causality as the organising idea.**

2. Why distributed-system debugging is difficult

Debugging one program is hard; debugging several programs that talk over an unreliable network is harder because the usual debugging assumptions stop holding. Eight difficulties drive every design choice in CHRONOS.

#	Difficulty	Plain explanation	What CHRONOS does about it
1	No shared clock	Each machine has its own clock, and they disagree	Orders events by cause (logical time), not by wall-clock time
2	Unpredictable message delivery	Messages can be late, lost, duplicated or arrive out of order	Owns the network, so every delivery is a recorded decision

#	Difficulty	Plain explanation	What CHRONOS does about it
3	Concurrency	Many things happen at once; the order differs run to run	Runs events one at a time in a recorded, replayable order
4	Partial failure	One node can die or freeze while the others carry on	Makes crashes, pauses and partitions first-class, repeatable faults
5	Heisenbugs	The bug disappears when you add a print statement or a breakpoint	Recording happens outside the app, so watching does not change behaviour
6	Cause and symptom are far apart	The bug appears on node X, long after and far from the real cause on node Y	Tracks the causal graph so we can walk from symptom back to cause
7	No rewind	You cannot step backwards through a run that already happened	Reconstructs state at any past point; replays deterministically
8	No what-if	You cannot cheaply test what would have happened with one change	Branches the timeline and runs counterfactual experiments

A small example of difficulty 1

Node A logs *10:00:05.200 – sent request*. Node B logs *10:00:05.150 – received request*. Read naively, B received the request **before** A sent it. That is impossible; B's clock is simply 100 ms ahead of A's. With only wall-clock timestamps, every cross-machine timeline is unreliable.

A small example of difficulties 3 and 5

RE fails only when the heartbeat from `p` is delayed past 300 ms *and* a client write arrives at `b` inside the same few milliseconds. Run it again and the timing is different, so the bug does not appear. Add a log line and the timing changes again. Production teams call this a *flaky* or *Heisenbug* failure, and it is the main reason developers lose days on distributed bugs.

The key insight

Almost all of this pain comes from **uncontrolled nondeterminism**: the clock, the network, the scheduler and the random numbers decide things that we cannot see or repeat. If a system owns those four things, it can record every decision and replay them. That is the idea CHRONOS is built on (Section 3).

3. CHRONOS's core idea

CHRONOS runs a distributed application inside a world it fully controls, turns the run into structured data, and then lets us navigate, replay, branch and compare that data. Think of it as **Git for executions**: every run is a commit-like timeline, you can check out any moment, create a branch where you change one thing, and diff two branches.

Plain idea

Imagine a board game played by several players. Normally the dice, the move order and the delays between turns are decided by chance. Now imagine a referee who rolls every die, calls every turn, and writes each decision in a notebook. With the notebook you can replay the whole game identically, stop at any turn, or say: *what if turn 12 had gone to a different player?* and play on from there.

In CHRONOS, the referee is the **controlled runtime**. The players are the application's nodes. The notebook is the **event log**.

The five moves

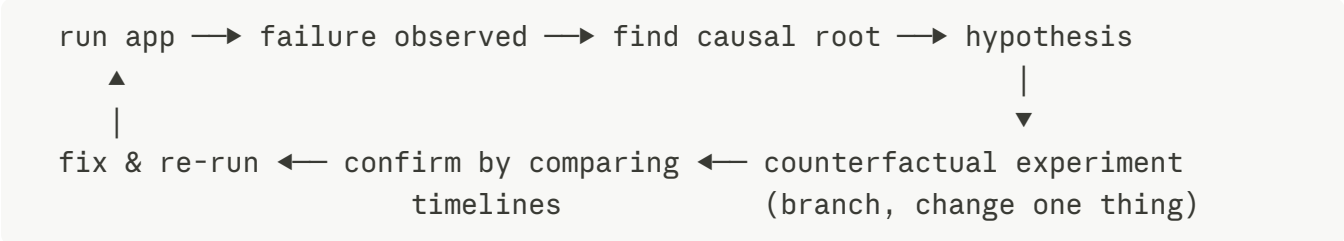
1. **Record.** The runtime logs every meaningful happening (message sent, message delivered, timer fired, fault injected, state changed) as an immutable *event*, stamped with logical clocks and causal links.
2. **Understand.** Events form a *causal graph* that shows what could have influenced what.
3. **Rewind and replay.** Any past moment can be inspected, and the whole run can be re-executed with identical results (deterministic replay).
4. **Branch and change.** From any moment, create an alternate timeline, inject a fault or alter a delivery, and run forward (counterfactual execution).
5. **Compare and explain.** Compare two timelines, find the first causal divergence, and trace it to the failure.

Why a controlled runtime is the right foundation

To replay a run you must be able to repeat every nondeterministic choice. Doing that for arbitrary Java programs means intercepting threads, clocks, the network, class initialisation and native code. That is a research-lab problem. CHRONOS instead gives the application a small **SDK** (the *Node API*) and asks it to do its time, networking, timers and randomness *through* the runtime. In exchange, CHRONOS can control logical time, event ordering, event delivery, deterministic randomness, scheduling, fault injection and replay. This is a deliberate architectural decision (D-01), not a limitation to hide.

The CHRONOS loop

This is the workflow a developer follows, and the story every demonstration tells:



4. Project goals

CHRONOS v1 succeeds if a developer can take a failing run of a distributed Java application on our runtime, find the earliest causal point of failure, and prove it with a counterfactual experiment, and if both of us can explain every mechanism involved.

Engineering goals

ID	Goal	How we will know
G1	**Record** every run as structured, causally linked, persisted events	Event log round-trips from disk; every event has clocks and causal parents
G2	**Deterministic replay**: same inputs and decisions give a byte-identical event log	Replay of 1,000 seeded runs yields identical logs
G3	**Time travel**: inspect any node's state, in-flight messages and timers at any past event	Seek to any event of a 100k-event timeline in under one second

ID	Goal	How we will know
G4	**Branching**: create alternate timelines that share history with their parent	Branch prefix is provably identical to parent prefix
G5	**Fault injection**: repeatable drops, delays, duplicates, partitions, crashes, pauses, clock skew	Same fault plan and seed always produce the same faulty run
G6	**Counterfactuals**: re-run from a past point with one change and report whether the failure persists	Planted bugs are confirmed or refuted correctly
G7	**Explanation**: compare two timelines and name the first causal divergence and the path to the failure	The detector names the planted root cause in every known-answer scenario
G8	**Visualisation**: show history, causal arrows and divergences	Space-time diagram with a highlighted divergence cone

Acceptance demonstrations

CHRONOS v1 is finished when all six of these can be run live from the command line on the flagship RE application.

ID	Demonstration	Shows
AD-1	Run RE with a failing seed; CHRONOS reports *single-primary invariant violated* and prints the causal chain from partition to violation	Recording, causality, invariants
AD-2	Replay the run and show identical event-log hashes; then run a deliberately nondeterministic node and watch CHRONOS flag the first mismatching event	Determinism and its verification
AD-3	Jump to the event just before the violation; show all node states and in-flight messages; step backward three times	Time travel, state reconstruction
AD-4	Branch before the partition, heal it 200 ms earlier, run forward, and show the violation disappears	Branching, faults, counterfactuals

ID	Demonstration	Shows
AD-5	Compare the failing and healed timelines; CHRONOS names the partition event as the first causal divergence	Comparison and divergence detection
AD-6	Open the generated static HTML report: lanes per node, causal arrows, divergence cone highlighted	Visualisation

Learning goals

The project is also a course in distributed systems. By the end, each of us must be able to explain, without notes, Lamport and vector clocks, happens-before, consistent cuts, event sourcing, deterministic replay, why identical code can behave differently, and what a counterfactual can and cannot prove. Every component section in this document carries a short teaching note for that reason.

Quality goals

Every subsystem ships with tests and a runnable demonstration before the next subsystem starts. Every mechanism that is the intellectual heart of CHRONOS (clocks, causal graph, scheduler, replay, branching, comparison) is implemented by us, not hidden behind a library.

5. Explicit non-goals

CHRONOS will not be a universal debugger. The boundaries below keep the project finishable by two students and keep the core idea sharp. If a feature request falls under a non-goal, the answer is no unless this section is formally amended.

Non-goal	Why we exclude it	What we do instead
Transparent record and replay of arbitrary Java applications	Needs control over threads, clocks, I/O, native code and class loading; research-grade effort	Applications are written against the CHRONOS Node API (D-01)

Non-goal	Why we exclude it	What we do instead
Replaying real multi-threaded races inside one node	Thread interleavings inside a JVM are not controllable without bytecode-level tools	Each node is a single-threaded event handler; concurrency is *between* nodes (D-02)
Production monitoring or tracing of live systems	Different problem (low overhead, sampling, scale)	CHRONOS is a laboratory and debugger, not an observability platform
Bytecode instrumentation, Java agents, JVMTI	Fragile and hides the mechanisms we want to learn	Explicit SDK calls
Exhaustive model checking or formal proofs	Different discipline (state-space explosion, formal specs)	Targeted counterfactual runs and seeded search campaigns
Proving the root cause of a bug	Causality in real systems is partly a judgment	We find the first *causal divergence* and test it with a but-for experiment
Byzantine (malicious or arbitrary) faults	Large extra semantics for little demo value	Crash, pause, drop, delay, duplicate, reorder, partition, clock skew
Running CHRONOS itself as a distributed system	Adds the hard problems we are studying to our own tool	CHRONOS runs on one machine; nodes are simulated
Merging timelines	No sound semantics for merging two executions	Timelines are compared, never merged
Support for languages other than Java	Splits effort	Java 21 only
Huge scale	Not needed for the learning and demonstration goals	Design target: about 3 to 20 nodes and up to about 1,000,000 events per timeline
Polished product UI, user accounts, cloud hosting, multi-user collaboration	Does not serve the core idea	CLI first, then a static HTML report; a live local web viewer is Stretch (D-18)

Non-goal	Why we exclude it	What we do instead
Real wall-clock behaviour (NTP, real latency)	Real time is exactly the nondeterminism we remove	Virtual time controlled by the runtime

Boundary rule

A technology or subsystem is added only if it answers: *What problem does it solve for CHRONOS, and why can we not do it simply ourselves?* Spring Boot, Kafka, Redis, Kubernetes, cloud services and microservices all currently fail this test.

6. Core concepts

This glossary fixes the vocabulary used everywhere else. Each term is explained in plain words first; the sections named in the last column give the full technical treatment. Use these exact words in code, docs and conversation.

Term	Plain meaning	Tiny example (RE or ping-pong)	Full treatment
Distributed system	Several programs on different machines cooperating by sending messages	`client`, `p`, `b`	2
Node	One participant: a program with its own private state that reacts to inputs	`p`	9
Message	A package of data sent from one node to another	Heartbeat from `p` to `b`	9
Event	One recorded fact about what happened, immutable once written	`p` sent heartbeat #12	10

Term	Plain meaning	Tiny example (RE or ping-pong)	Full treatment
Step	One unit of work the runtime performs: pick one pending item, run one node handler to completion	Deliver heartbeat #12 to `b`	9, 10
Timeline	One complete, immutable recording of one execution	The failing run, T0	17
Virtual time	The simulated clock the runtime controls, in milliseconds	Heartbeat due at 300 ms	9, 11
Logical time	Numbers that order events by cause instead of by wall-clock	Lamport and vector clocks	11
Lamport clock	One counter per node; gives a consistent ordering but is not exact about causality	`L = 5`	11
Vector clock	One counter per node, kept as a list; tells exactly whether two events are causally related	`[3,3,0]`	11
Causality (happens-before)	Event X *could have influenced* event Y	Send before its receive	12
Concurrent events	Neither could have influenced the other	`p` handling a write while `b` handles a timer	12
Causal graph	Events as dots, causal links as arrows	The whole run drawn as a graph	13

Term	Plain meaning	Tiny example (RE or ping-pong)	Full treatment
State	Everything a node remembers	`p` data map, role, heartbeat counter	14
Snapshot (checkpoint)	A frozen copy of the whole world at one moment	World at event 5000	14
Consistent cut	A snapshot where no message is received that was not yet sent	Any prefix of the recorded order	14
Event sourcing	Storing the history of changes rather than only the latest state	Rebuilding `p`'s state from the log	14
Determinism	Same inputs and decisions always give the same outputs	Replay produces an identical log	15
Nondeterminism source	Anything that can differ run to run: clock, network, random, scheduling order	`System.currentTimeMillis()`	9, 15
Replay	Re-executing a recorded run so it unfolds identically	AD-2	15
Time travel	Moving a cursor through a recorded timeline and inspecting any moment	Jump to event 4521	16

Term	Plain meaning	Tiny example (RE or ping-pong)	Full treatment
Branch	A new timeline that shares history with a parent up to a chosen point	T0.1 branched at event 120	17
Fault	A controlled failure of the environment, recorded as an event	Partition between `p` and `b`	18
Intervention	A deliberate change applied to a branch	Drop the heartbeat at 300 ms	19
Counterfactual	A what-if answered by running the changed branch forward	Would the failure happen if the heartbeat had arrived?	19
Invariant	A rule that must always hold; breaking it is a failure	At most one primary at any time	19, 20, 21
Divergence	The point where two timelines stop matching	First event that differs	20
Decision source	The component that answers every nondeterministic question the runtime asks	Seeded policy, or the recorded log	9, 15

Two distinctions that prevent most confusion

1. **Reconstruction versus replay.** *Reconstruction* rebuilds a past state by reading the log; no application code runs. *Replay* re-executes the application code and checks that it reproduces the log. Reconstruction is for looking; replay is for verifying and for branching.

2. **Time versus order.** *Virtual time* says when something is scheduled in the simulation. *Logical time* says what could have influenced what. *Sequence number* (position in the log) says in what order the runtime happened to do it. These are three different things and CHRONOS records all of them.

7. Functional requirements

These are the things CHRONOS must do. Each has a priority (Must, Should, Could, Stretch) and the phase of Section 24 in which it is built. Later sections explain how each is met.

ID	Area	Requirement	Priority	Phase
FR-01	Runtime	Applications define nodes through the Node API: handlers for start, message and timer; every outside effect goes through the node context	Must	3
FR-02	Runtime	A deterministic single-threaded scheduler runs one agenda item at a time in virtual time, with stable tie-breaking	Must	3
FR-03	Runtime	A simulated network delivers messages with configurable per-link latency; the core uses no real sockets	Must	3
FR-04	Runtime	Timers can be set and cancelled in virtual time	Must	3
FR-05	Runtime	Seeded deterministic random streams per node and per runtime purpose, using our own generator	Must	3
FR-06	Runtime	A scenario declares nodes, configuration, workload, fault plan and invariants; it runs by name with a seed and a horizon	Must	3–4

ID	Area	Requirement	Priority	Phase
FR-07	Events	Every effect is recorded as an event with a deterministic identity (node and per-node counter; events on env are keyed by what they concern, D-24)	Must	4
FR-08	Events	Each event carries virtual time, Lamport value, vector clock and causal parents with edge kinds	Must	4–5
FR-09	Events	Events are immutable and the log is append-only	Must	4
FR-10	Events	The events of one step are appended as one atomic group through a pluggable sink	Must	4, 7
FR-11	Clocks	Lamport clocks follow the send and receive rules on every node	Must	5
FR-12	Clocks	Vector clocks support comparison returning before, after, equal or concurrent	Must	5
FR-13	Causal graph	Build the causal graph from a live event stream or from a stored log	Must	6
FR-14	Causal graph	Queries: causal past, causal future, happens-before, concurrent, causal path	Must	6
FR-15	Causal graph	Self-check that vector-clock answers agree with graph reachability	Should	6
FR-16	Persistence	Persist events to an append-only segmented log with a format version; reload it; tolerate a torn final record	Must	7
FR-17	Persistence	A timeline catalog stores metadata, parent and branch point	Must	7, 11
FR-18	Persistence	Indexes by sequence, node, kind and virtual time	Must	7

ID	Area	Requirement	Priority	Phase
FR-19	State	Node state is canonically serialisable; a digest is recorded after every step	Must	8
FR-20	State	Reconstruct any node's state, the in-flight messages and the pending timers at any position by reading the log only	Must	8
FR-21	State	World checkpoints at a configurable interval and at branch points; a runnable world can be restored from one	Must	8
FR-22	Replay	Replay a timeline from the start or from a checkpoint with every decision taken from the log	Must	9
FR-23	Replay	Verify each regenerated event against the recording; report the first mismatch as a determinism violation	Must	9
FR-24	Replay	Stepped replay with breakpoints on event predicates, virtual time, node or invariant violation	Should	10b
FR-25	Time travel	A debug session with a cursor: goto, step, back, next and previous event on a node, find	Must	10a, 10b
FR-26	Time travel	Inspect states, in-flight messages, timers and clocks at the cursor	Must	10a
FR-27	Time travel	Causal queries from the cursor: why did this happen, what did it cause	Must	10a, 10b
FR-28	Time travel	State watch: find the first or last event at which a state predicate changed	Should	10b
FR-29	Branching	Create a branch at any step boundary of any timeline, sharing the prefix by reference	Must	11
FR-30	Branching	List the timeline tree; inspect timelines; delete leaf timelines	Must	11

ID	Area	Requirement	Priority	Phase
FR-31	Faults	Fault plans with triggers and actions: partition and heal at a virtual time (Phase 12a, Must); drop, delay, crash (Phase 12b, Must); restart, duplicate, reorder, pause, clock skew (Phase 12b, Should)	Must	12a, 12b
FR-32	Faults	Faults are recorded as events with causal edges to the events they affect, and OMISSION edges to the timer firings that noticed the silence they caused (D-23)	Must	12a, 12b
FR-33	Faults	Probabilistic faults are decided by identity-keyed hashing so they stay stable across branches	Should	12b
FR-34	Counterfactual	Apply interventions at a branch point and run to the horizon or to a violation	Must	13a, 13b
FR-35	Counterfactual	Report verdict (failure persists or disappears), new timeline and divergence summary	Must	13a, 14a
FR-36	Counterfactual	Run independent branches in parallel, each in an isolated runtime	Should	13b
FR-37	Counterfactual	Seed campaign: run many seeds or fault plans and list the violating timelines	Should	12b
FR-38	Counterfactual	Automated intervention search across the causal past of a violation to find critical events	Stretch	Unscheduled
FR-39	Comparison	Align events across two timelines by identity and compare content digests	Must	14a
FR-40	Comparison	Find each node's first local divergence and classify differences as missing, extra, changed or reordered	Must	14a

ID	Area	Requirement	Priority	Phase
FR-41	Comparison	Compute the causal root divergences and report the first causal divergence (the earliest root; taint is computed per timeline, D-16)	Must	14a
FR-42	Comparison	Produce the explanation path from the root divergence to the failure	Must	14a
FR-43	Comparison	Flag suspected application nondeterminism: same inputs, different outputs	Should	14b
FR-44	Invariants	Global invariants are checked after every step; a violation is recorded as an event and may stop the run; from Phase 8 it carries WITNESS edges to the last writers of the state fields the invariant read (D-17)	Must	4, 8
FR-45	Visualisation	Text export of a space-time diagram first; later DOT and JSON exports	Must	6, 15a
FR-46	Visualisation	Static, self-contained HTML report generated by the CLI: inline SVG space-time diagram with node lanes and causal arrows, event table, causal path, divergence highlight, optional hover and filter; no network calls (D-18)	Must	15a
FR-47	Interface	Command-line interface covering run, replay, inspect, branch, inject, compare, grown phase by phase	Must	4–15a
FR-48	Quality	One command runs a scenario twice and compares log hashes (determinism check)	Must	4
FR-49	Demos	Demo applications: ping-pong, a causal-ordering demo, RE with a seeded bug, plus at least two further known-answer bug scenarios	Must	3–15a

ID	Area	Requirement	Priority	Phase
FR-50	Visualisation	Live local web viewer on the JDK HTTP server with a time slider (post-v1)	Stretch	Unscheduled
FR-51	Counterfactual	Each intervention is marked by an INTERVENTION_APPLIED event on env at the branch point, with id iv: <interventionId>; origin is part of the content digest (D-25)	Must	13a
FR-52	Causal graph	OMISSION edges link a fault event to a timer firing on a node the fault touches, when the fault was active during the timer's wait (D-23); partition scope in 12a, crash and pause scopes in 12b	Must	12a, 12b

8. Non-functional requirements

These describe how well CHRONOS must do its job. Numeric targets are *initial planning targets*; Phase 16 measures reality and we revise them honestly rather than hiding misses.

ID	Category	Requirement	Target
NFR-01	Determinism	The same scenario, seed, fault plan and code version produce a byte-identical event log	100% over 1,000 seeded runs of each demo scenario
NFR-02	Correctness	Clock and causality rules hold on every recorded event	Vector-clock order equals graph reachability on all test timelines
NFR-03	Fidelity	State reconstructed from the log equals the state seen live during the run	Exact match of state digests at every step
NFR-04	Immutability	Recorded timelines are never modified; any change creates a branch	Verified by a test that checks file hashes before and after sessions

ID	Category	Requirement	Target
NFR-05	Recording speed	Recording must not make runs painfully slow	At least 50,000 events per second in memory; at least 20,000 per second persisted (initial)
NFR-06	Seek latency	Jumping to any event in a long timeline is interactive	Under 1 second for 100,000 events; under 5 seconds for 1,000,000
NFR-07	Replay speed	Replay is not much slower than the original run	Within 2 times the original run time
NFR-08	Branch cost	Creating a branch does not copy the parent history	Branch creation time independent of prefix length apart from checkpoint restore
NFR-09	Scale	Support the design envelope	3 to 20 nodes and up to 1,000,000 events per timeline
NFR-10	Storage	Keep the log compact enough to handle many runs	Measure and report bytes per event; no pass-or-fail target (initial expectation: under 300 bytes on average without full states)
NFR-11	Durability	A crash while writing leaves a readable log	Torn last record is detected and discarded, not propagated
NFR-12	Testability	Every component can be tested in isolation	Each component depends only on interfaces of the components below it
NFR-13	Maintainability	Clear module boundaries and a single direction of dependency	Enforced by a build-time dependency rule
NFR-14	Portability	Runs on the team's operating systems	Linux, Windows and macOS with Java 21 and a standard build tool

ID	Category	Requirement	Target
NFR-15	Usability	A new team member can run the flagship demo from the README	Under 10 minutes, three commands
NFR-16	Explainability	Output speaks in CHRONOS vocabulary and always links a claim to events	Every report cites event IDs
NFR-17	Learnability	Every component has a teaching note and a runnable demo	Checked at each phase review
NFR-18	Format evolution	Stored files carry a format version	Old logs are readable, or refused with a clear message
NFR-19	Dependency discipline	Minimal third-party dependencies, each justified in the decision log	Target: fewer than 5 runtime libraries (Section 23)
NFR-20	Determinism hygiene	Core and demo packages contain no use of banned nondeterministic APIs	Automatic build-time scan for wall-clock time, unseeded randomness and raw threads

9. Controlled execution model

CHRONOS applications run inside a deterministic simulation: nodes are single-threaded event handlers, and only the runtime decides which thing happens next, when it happens, and what the network does to it. This is the foundation every other section stands on.

Plain idea

Picture a puppet theatre. The nodes are puppets. The runtime is the puppeteer holding every string. A puppet cannot look at a real clock, only at the clock on the stage. It cannot post a letter, only hand a card to the puppeteer, who decides when it arrives. Because the puppeteer controls everything, the show can be performed again exactly.

Tiny example

`p` wants to send a heartbeat to `b`. It calls `send(b, Heartbeat)` on its node context. Nothing touches a socket. The runtime records the send, picks a delivery time (current virtual time plus a latency) and places the delivery in the **agenda**. Later the scheduler picks that item, moves virtual time forward, and calls `b`'s message handler.

Building blocks

Concept	Meaning
Node	An object implementing the Node API: a handler for start, one for each incoming message, one for each timer (and one for recovery after a crash, from Phase 12b)
Node state	All of a node's memory, held in one serialisable state object with a volatile zone (lost on a crash) and a durable zone (survives a crash, standing for disk); the node declares a state codec through the Node API (D-04)
Node context	The only door to the outside world: send, set and cancel timers, read virtual time, draw random numbers, read configuration, annotate the log
World	The whole simulation: all nodes, the agenda, the network model, the virtual clock, the random streams, the fault state
Agenda	The set of pending items: message deliveries, timer firings, scheduled client inputs, scheduled fault actions; each has a due virtual time and a stable key
Scheduler	Picks the next agenda item and runs it as one step
Step	One scheduler action: take one item, run one handler to completion, collect its effects
Virtual clock	The simulated time in milliseconds; it jumps to the due time of the next item, so waiting costs nothing

Concept	Meaning
Network simulator	Holds in-flight messages, applies latency, partitions and other faults
Decision source	Answers every nondeterministic question the engine asks (below)

The step cycle

1. The scheduler takes the agenda item with the smallest (due time, stable tie-break key).
2. Virtual time advances to that due time.
3. The fault injector may transform the item (drop it, delay it, duplicate it); any such action is recorded as a fault event.
4. The trigger event (message delivered, timer fired, client input, node start) is recorded and the clocks are updated.
5. The node's handler runs to completion. It cannot be interrupted and no other node runs meanwhile.
6. The handler's effects (sends, timer changes, random draws) are collected. Each becomes a recorded event, and new agenda items are created with due times supplied by the decision source.
7. The runtime computes the state digest of the node and records the state after the step. From Phase 8 it also compares per-field digests to update the last-writer map (D-17).
8. All events of the step are appended to the recorder as one atomic group.
9. Invariants are checked, reading node state through a recording accessor (D-17). A violation is recorded as an event, and the run stops or continues according to configuration.
10. Repeat until the agenda is empty, the horizon is reached, or a stop condition fires.

Stable tie-breaking

When two items are due at the same virtual time, the order comes from a **stable key** (due time, target node index, event-kind rank, message identity), never from insertion order or object hash codes. Why this matters: in an alternate timeline, an unrelated extra item must not reshuffle everything around it. Stable ordering is what makes counterfactual results meaningful (Section 19).

The decision source: one seam for all nondeterminism

The engine never decides anything by itself. It asks a **decision source**: which item runs next when there is a choice, what latency this message gets, whether this message is dropped, what value this random call returns, whether a fault rule fires. There are three implementations of the same interface:

- **Record mode:** answers come from a seeded policy and the fault plan; every answer is written into the event it affects.
- **Replay mode:** answers are read back from the recorded events.
- **Branch mode:** answers come from the parent's log up to the branch point, then from the policy plus the interventions.

The same engine runs in all three modes, so replay correctness is a property of one engine, not of a second code path (D-19). There is **no separate decision log**: each decision is stored as a field of the event it influenced (a send event stores its delivery time or its drop; a random draw is an event with its value).

Where the three sources live (D-19). The decision-source interface and the record-mode source belong to the runtime. The replay and branch sources read stored logs, so they live in the analysis layer and reach the log only through the LogReader interface of the model module. The engine sees nothing but the interface.

Identity-keyed randomness (D-06)

Plain idea: instead of one shared dice rolled in order, every question gets a die decided by who is asking. *Example:* the latency of message `p#17` is computed from `hash(seed, p#17)`, not from the next number in a shared stream. If an alternate timeline differs earlier, message `p#17` still gets exactly the same latency. Each node also has its own independent random stream, derived from the seed and the node identity.

Why: with one shared stream, one extra random call anywhere shifts every later value everywhere, creating false differences between timelines. We call that *butterfly noise*, and this rule removes it.

CHRONOS uses its own small random generator whose entire state is one number. That makes it snapshot-able and identical on every JVM; the JDK generator hides its state.

Determinism contract for application authors

Application code is replayable only if it obeys this contract. It is enforced by a build-time scan (NFR-20) and, as a safety net, by replay verification (Section 15).

Do not use	Why it breaks replay	Use instead
Wall-clock time (system time, current instant)	Differs every run	Virtual time from the context
Unseeded randomness (JDK random classes, random UUIDs)	Differs every run	Random draws from the context
Threads, executors, futures, blocking waits inside node code	Interleaving is uncontrolled	Messages and timers
Sleeping	Real waiting is meaningless in virtual time	A timer
Real sockets, files, HTTP	Outside the controlled world	Context send; the durable zone of the node state (Phase 12b)
Hash-based collections of objects whose hash code is identity-based (this includes enums and objects without a custom hash code)	Iteration order can change between JVM runs	Ordered or tree-based collections, or enum-keyed maps
Static or shared mutable state between nodes	Hidden channel the runtime cannot see	State only in the node state object
Default object-to-string, identity hash, locale or time-zone dependent formatting	Output varies by machine	Explicit formats

Concurrency stance: three alternatives

Option	Idea	Advantages	Disadvantages
A. Deterministic discrete-event simulation	One engine thread; nodes are event handlers	Simplest to make deterministic; fast; easy replay; easy to snapshot	Node code cannot use threads or blocking calls

Option	Idea	Advantages	Disadvantages
B. Baton-passing threads	Each node has its own thread, but a scheduler lets exactly one run at a time	Node code can block naturally	Thread switching cost; harder snapshots; subtle deadlocks
C. Free threads plus recorded order	Real parallel threads; record the interleaving	Closest to real programs	Needs deep JVM-level control; exactly what we declared a non-goal

Recommendation: A (D-02). Concurrency still appears where it genuinely belongs: an asynchronous event writer, parallel counterfactual runs (each in its own isolated world), and stress tests of the recorder and store (Section 23).

Lifecycle and failure semantics

- A node is NEW, then RUNNING; it can be PAUSED (frozen, messages queue up) or CRASHED (silent). From Phase 12b a crashed node can restart.
- Deliveries to a crashed node are recorded as dropped with the reason *target down*. Its timers are discarded.
- A crash wipes the node's volatile state and keeps only its durable zone (Phase 12b). Before then (Phases 3 to 11) a node has only its volatile zone, and a crash is simply stop.
- An exception escaping a handler is recorded as a node-failed event and, by default, stops the run, since it is probably an application bug.
- A run ends when the agenda is empty, a virtual-time horizon is reached, or an invariant violation stops it.

Run modes

The core is **simulation mode** only (D-03): all nodes live in one JVM and the network is simulated. A *live mode* with separate processes over real TCP could record but could not replay deterministically; real-network execution is out of scope for v1. A live-network recorder is a possible post-v1 idea and must never enter the deterministic replay core.

Inputs and outputs

Consumes: scenario, seed, fault plan, decision source. **Produces:** an ordered stream of events to the recorder, and the final world. **Used by:** recorder, replay engine, branching, counterfactual engine. **Tested by:** determinism tests (run twice, compare logs), scheduler unit tests, and golden scenarios (Section 25).

10. Event model

An event is an immutable record of one thing that happened in a run, and the event log is the single source of truth from which every other CHRONOS view (graph, state, replay, comparison) is built.

Plain idea

An event is one line in a very strict diary: *who* did *what*, *when* (in three senses of time), and *because of what*. Once written, a line is never edited.

Tiny example: ping-pong

Nodes A and B. A sends `ping` to B; B replies `pong`. Latency is 10 ms. The full recording is six events:

seq	Event id	Node	Kind and detail	Virtual time (ms)	Lamport	Vector [A,B,env]	Parents
0	A#1	A	NODE_STARTED	0	1	[1,0,0]	none
1	B#1	B	NODE_STARTED	0	1	[0,1,0]	none
2	A#2	A	MESSAGE_SENT ping	0	2	[2,0,0]	A#1
3	B#2	B	MESSAGE_DELIVERED ping	10	3	[2,2,0]	B#1, A#2
4	B#3	B	MESSAGE_SENT pong	10	4	[2,3,0]	B#2
5	A#3	A	MESSAGE_DELIVERED pong	20	5	[3,3,0]	A#2, B#3

Read it like this: `B#2` has two parents because B's delivery depends both on B's own previous event (`B#1`) and on the send (`A#2`). The clock columns are explained in Section 11. The last vector slot belongs to env, which never acts in ping-pong, so it stays 0.

Anatomy of an event

Field	Meaning
`eventId`	Node name plus a per-node counter, such as `A#2`. **Deterministic identity**: the same logical happening gets the same id in every timeline. This is the key used to compare timelines. The exception is env: its events are keyed by what they concern, never by a counter (D-24), for example drop: <messageId>, fault:<ruleId>#<n>, iv: <interventionId> and inv:<name>#<n>
`seq`	Position in this timeline's log. Assigned only by the recorder, strictly increasing. A branch continues numbering after its branch point
`timelineId`	The timeline this record belongs to
`nodeId`	Where it happened. Environment-level facts (partition, drop) belong to a pseudo-node called `env`, which also holds a slot in vector clocks and follows the same Lamport and vector rules as a node
`kind`	One of the kinds below
`virtualTime`	Simulated milliseconds when it happened
`lamport`, `vector`	Logical time stamps (Section 11)
`parents`	List of (parent event id, edge kind): the causal links (Section 12)
`payload`	Kind-specific data in canonical form: message type and body, timer id, random value, fault description
`stateDigestAfter`, `stateAfter`	Present on a step's trigger event: a digest of the node state after the step, and the full state when it changed (D-10); from Phase 8 the stored state is split into a volatile zone and a durable zone
`origin`	Optional: the fault rule or intervention that caused the event; it is part of the content digest (D-25)

Event kinds

Kind	Produced when	Key payload	Extra parents beyond the node's previous event
NODE_STARTED	Node begins	Initial state digest	none
NODE_CRASHED, NODE_RESTARTED	Crash or recovery	Reason	The fault event that caused it
NODE_PAUSED, NODE_RESUMED	Freeze and thaw	Duration	The fault event
NODE_FAILED	A handler threw an exception	Exception summary	The trigger event
MESSAGE_SENT	A handler calls send	Message id, destination, type, body, scheduled delivery time	The step's trigger event
MESSAGE_DELIVERED	Runtime hands a message to a node	Message id	The matching send event
MESSAGE_DROPPED	Network discards a message (on `env`)	Message id, reason	The send event, and the fault event if any
MESSAGE_DUPLICATED	Network creates a second delivery (on `env`)	Message id	The send event, the fault event
TIMER_SET, TIMER_CANCELLED	Handler sets or cancels a timer	Timer id, due time	The step's trigger event

Kind	Produced when	Key payload	Extra parents beyond the node's previous event
TIMER_FIRED	Timer reaches its due time	Timer id	The matching set event; plus an OMISSION edge from each fault event that was active during the timer's wait and touches this node (D-23)
CLIENT_INPUT	External workload item reaches a node	Input body	none (external)
RANDOM_DRAWN	Handler asks for a random value	Stream, value	The step's trigger event
FAULT_INJECTED, FAULT_CLEARED	A fault rule fires or ends (on `env`)	Rule id, action, parameters	The trigger that fired the rule
INVARIANT_VIOLATED	A global rule breaks (on `env`)	Invariant name, witness events	The step's trigger event (STEP_EFFECT), plus one WITNESS edge per witness (D-17)
APP_NOTE	Application labels something for readability	Free text	The step's trigger event; no causal effect
INTERVENTION_APPLIED	A branch applies an intervention at its branch point (on env; id iv: <interventionId>)	Intervention id, type, parameters	The previous env event of the shared prefix, if any. It is the parent (FAULT edge) of every event the intervention changes directly

Steps and events

A **step** (Section 9) produces one **trigger event** (a delivery, a timer firing, a client input, a start) followed by **effect events** (sends, timer changes, random draws, notes). They are appended together as one atomic group, so a crash of CHRONOS itself can never leave half a step in the log.

Identity and ordering rules

1. `eventId` is the node name plus a counter that starts at 1 and increases by one for each event on that node. Events on env are the exception: they are keyed by what they concern (drop:<messageId>, fault:<ruleId>#<n>, iv:<interventionId>, inv:<name>#<n>), so one extra env event in a branch never renumbers the others (D-24). The env counter feeds the clocks only, never identity.
2. A message's id is the id of its send event. A timer's id is the id of its set event.
3. The log order is a *linear extension of causality*: every parent has a smaller `seq` than its child.
4. Events are never modified. Corrections are new events or new timelines.
5. The same `eventId` in two timelines means *the same logical happening*; its content may differ, and that difference is exactly what comparison looks for.
6. A **content digest** is a hash of node, kind, canonical payload, parent ids, origin and state digest. It excludes `seq` and `timelineId`. A strict variant also includes virtual time. Origin is included on purpose (D-25): an event produced by an intervention is never Same as an otherwise identical event produced by the baseline.

Granularity: three alternatives

Option	Advantages	Disadvantages
Every method call or field write	Maximum detail	Impossible without instrumentation; huge volume
Only message sends and receives	Simple	Misses timers, randomness, state, faults
Handler-level steps with effect events	Matches the controlled runtime exactly; small; captures every nondeterministic decision	Does not show what happens *inside* a handler

Recommendation: handler-level (D-05). To see inside a handler, replay to that step and use an ordinary debugger.

Message bodies and storage

The body is stored once, in the send event. The delivery event stores only the message id. Bodies use the same canonical encoding as node state (D-09).

Inputs and outputs

Produced by: the runtime through the recorder. **Consumed by:** store, causal graph, state reconstructor, replay verifier, time-travel session, comparator, visualiser. **Tested by:** golden event logs for ping-pong and RE; property tests for the ordering rules.

11. Logical time model

Because nodes share no real clock, CHRONOS orders events by cause using two logical clocks, Lamport and vector, and keeps them separate from virtual time and from log position.

Plain idea

Three friends in different cities send each other letters and have no common watch. Each keeps a private counter of what they have done. When they post a letter, they write their counter on it. When they receive one, they set their own counter higher than both their own and the one on the letter. Now any number tells you something true: *at least this many things happened before this moment*.

Tiny example: the Lamport clock

In the ping-pong trace of Section 10, the Lamport column reads 1, 1, 2, 3, 4, 5. B′s receive of `ping` jumps to 3 because it must exceed both B′s own counter (1) and the number on the message (2).

Lamport clock: the rules

1. Every node starts with counter 0.
2. **Local event** (including start): counter increases by 1; the event is stamped with it.
3. **Send:** counter increases by 1; the send event is stamped with it and the message carries it.

4. **Receive:** counter becomes the larger of (own counter, counter on the message), plus 1; the receive event is stamped with it.

What it guarantees: if event X could have influenced event Y, then Lamport(X) is smaller than Lamport(Y). **What it does not guarantee:** the reverse. A smaller number does *not* prove influence. To produce one total order for display, ties are broken by node index.

Tiny example: where Lamport misleads

Node A does one local event (Lamport 1). Node C does three local events (Lamport 1, 2, 3). Nobody sends anything. A's event has a smaller number than C's third event, yet they are completely unrelated. Lamport alone cannot tell *before* from *concurrent*. Vector clocks can.

Vector clock: the rules

Plain idea: instead of one counter, each node keeps a tally for everyone: how many events of A, of B, of C it knows about. Comparing two tallies shows for certain whether one event could have influenced the other.

A vector has N + 1 slots for N nodes (one per node in the order the scenario declares them, plus one last slot for `env`). Node *i* owns slot *i*.

1. Every node starts with all slots 0.
2. **Local event:** slot *i* increases by 1.
3. **Send:** slot *i* increases by 1; the message carries a copy of the whole vector.
4. **Receive:** take the slot-by-slot maximum of own vector and the message's vector, then slot *i* increases by 1.

Comparing: vector V1 is *less than* V2 if every slot of V1 is less than or equal to the matching slot of V2 and the two are not equal. Then event 1 **happened before** event 2. If neither is less than the other, the events are **concurrent**.

Worked example with three nodes

Event	What happens	Lamport	Vector [A,B,C,env]
a1	A local event	1	[1,0,0,0]
a2	A sends m1 to B	2	[2,0,0,0]
c1	C local event	1	[0,0,1,0]

Event	What happens	Lamport	Vector [A,B,C,env]
b1	B receives m1	3	[2,1,0,0]
b2	B sends m2 to C	4	[2,2,0,0]
c2	C receives m2	5	[2,2,2,0]

Questions the vectors answer exactly:

- `a1` and `c2`: [1,0,0,0] is less than [2,2,2,0], so `a1` happened before `c2` (through m1 then m2).
- `a2` and `c1`: [2,0,0,0] and [0,0,1,0] are incomparable, so they are **concurrent**, even though Lamport gives 2 and 1 and would tempt us to say `c1` came first.

The four ideas of time in CHRONOS

Clock	Answers the question	Do not use it for	Stored on every event
Virtual time (ms)	When is this scheduled in the simulation? Timers, latencies, timeouts	Deciding cause: unrelated events can share a time, and related events can be far apart	Yes
Lamport	A cheap consistent ordering for display and tie-breaking	Deciding whether two events are related	Yes
Vector	Exactly: did X happen before Y, or are they concurrent? Also consistent cuts	Telling how much *real* time passed	Yes
Sequence number	Where is this event in the recorded order? Cursor position	Cause: it is just one of many valid orders	Yes

Why both Lamport and vector (D-07)

Vector clocks alone are logically enough. We keep Lamport as well because it costs one number, it is the natural first thing to learn and test, and it gives a simple display order. The vector's cost is one number per node on every event and message; with up to 20 nodes

this is acceptable, and compression (storing the difference from the previous local event) is an optimisation for Phase 16, not a v1 requirement.

Membership and the env slot

The node set is fixed for a run, so the vector length is the number of nodes plus one for `env`. In the ping-pong examples `env` never acts, so its slot stays 0, but it is still shown: every table, diagram, golden file and stored event lists all N + 1 slots (D-07). Nodes joining mid-run are out of scope for v1; the extension would be sparse (map-based) vectors.

Clocks on every edge kind. `env` follows exactly the same Lamport and vector rules as a node. Every edge kind (MESSAGE, FAULT, WITNESS and OMISSION) carries the clock values of its source event, just as a message does, and the target event merges them with the receive rule. So a violation event on `env` merges the vectors of all its witnesses before it increments its own slot, and laws M3 and M4 hold for fault, witness and omission links too.

Clock skew and logical time

A clock-skew fault changes only the time a node *reads* from its context. It never changes Lamport or vector values, because those count causal steps, not time. This is why logical clocks stay trustworthy when physical clocks lie.

Inputs and outputs

Consumes: send and receive hooks from the runtime. **Produces:** a stamp (Lamport, vector) for every event and a clock value attached to every message. **Depends on:** nothing, which makes it the easiest first component to test. **Tested by:** property tests (if X could influence Y then Lamport(X) < Lamport(Y); vector order equals graph reachability; merge is commutative, associative and idempotent), the worked examples above as fixed tests, and random message-passing runs checked against a brute-force reachability computation.

12. Causality model

CHRONOS defines causality as *happens-before*: event X happened before event Y if X could have influenced Y, and two events with no such link are concurrent. This is potential causality, so CHRONOS labels each causal link with a kind to tell certain links from merely possible ones.

Plain idea

Two things happening one after the other does not mean the first caused the second. In a distributed system, X can influence Y only if information can travel from X to Y, and information travels in exactly two ways: through a node's own memory (earlier events on the same node) and through messages. If neither route exists, the two events cannot affect each other, however close their timestamps are.

Tiny example from RE

`p` handles client write W7, then sends heartbeat #13 to `b`, which delivers it. W7 could have influenced what `b` did on receiving heartbeat #13, so W7 happened before that delivery. But `b` receiving the *earlier* heartbeat #12 and `p` handling W7 may be **concurrent**: no chain of memory or messages links them. A timestamp-sorted log would still show one before the other and invite a wrong story.

The happens-before relation

1. **Same node:** if X occurs before Y on the same node, then X happened before Y.
2. **Message:** a send happened before its matching receive.
3. **Transitivity:** if X happened before Y and Y happened before Z, then X happened before Z.
4. **Concurrent:** if neither X happened before Y nor Y before X, the events are concurrent.

These four rules cover memory and messages. The environment adds links of its own: a FAULT, WITNESS or OMISSION edge from X to Y also makes X happen before Y, and it carries clocks exactly as a message does (Section 11).

Potential versus certain causes

Rule 1 is generous: a node's later behaviour may depend on *any* earlier event on that node, because they share memory. So happens-before shows what *could* have mattered, not what *did*. CHRONOS keeps this honest by tagging each parent link with a kind (also used in Section 13):

Edge kind	From to	Meaning	Certainty
PROGRAM_ORDER	Previous event on the same node to this event	The node's memory links them	Potential

Edge kind	From to	Meaning	Certainty
STEP_EFFECT	A step's trigger event to each of its effect events	The handler ran *because of* the trigger	Certain
MESSAGE	Send event to the delivery event	The delivery exists *because of* the send	Certain
TIMER	Timer-set event to the timer-fired event	The firing exists because of the set	Certain
FAULT	Fault event to the event it altered	The drop, delay or crash is *because of* the fault	Certain
WITNESS	The last-writer event of a state field to the INVARIANT_VIOLATED event that read that field (D-17)	The violation was computed from what that event wrote	Certain
OMISSION	A fault event to a TIMER_FIRED event on a node the fault touches (D-23)	The timer fired because expected messages never came; the fault silenced them	Potential (followed by the default why)

Reachability and concurrency use **all** edges, because any of them could carry influence. Explanation paths in Section 20, and the default why command, prefer the **certain** edges and the OMISSION edges, which makes the story told to the developer shorter and more convincing.

Omission: when the cause is a message that never came (D-23)

Plain idea: sometimes the cause of an event is something that did not happen. A smoke alarm that rings because the usual all-clear signal stopped arriving was not triggered by any message.

In RE, a partition drops `p`'s heartbeats, so `b` receives nothing and its 300 ms watchdog fires. No message links the partition to the watchdog: a dropped message is never delivered, so it creates no MESSAGE edge into `b`. Without help, the graph would show no

path from the partition to the promotion, and AD-1 and AD-5 would lose their explanation.

CHRONOS therefore adds an **OMISSION edge** from a fault event to a TIMER_FIRED event when the fault was active at any time between the timer's set event and its firing, and the fault touches the node. In Phase 12a, "touches" means the node sits on a side of an active partition that cuts it off from another node. In Phase 12b, crash and pause faults count when the node has received messages from the crashed or paused node before. An OMISSION edge is *potential* (the timer might have fired anyway), it carries clocks like a message, it counts in reachability, and the default `why` follows it.

From causality to blame: suspects and trials

Happens-before cannot say which event *caused* a failure, but it removes most of the innocent ones. The **causal past** of a failure event is the set of suspects: nothing outside it could have contributed. In a run of 10,000 events the causal past of a violation might hold only a few hundred. A **counterfactual experiment** (Section 19) is then the trial: change a suspect and see whether the failure survives. This *suspects then trial* method is how CHRONOS turns an ordering relation into evidence.

What causality is not

- It is not time order. Concurrent events have no cause-and-effect order, though the log lists them in some order.
- It is not a proof. It bounds what could have mattered; experiments test what did.
- It is not about application-level meaning. CHRONOS knows messages and memory, not what the code intended.

Inputs and outputs

Derived from: the parent links recorded on each event and the vector clocks. **Used by:** causal graph, time-travel queries (why, effects), comparator (root divergences, explanation path), visualiser (arrows). **Tested by:** the textbook examples in Section 11, a rule that no parent link points forward in the log, and agreement between vector-clock comparison and graph reachability on every test timeline.

13. Causal graph

The causal graph is a directed acyclic graph with one vertex per event and one edge per causal link, built from the event log, so that questions like *what could have caused this?* are answered by walking arrows.

Plain idea

It is a family tree for events. Each dot is an event. Each arrow points from a cause to its effect. To find out why something happened, walk the arrows backwards; to find what it led to, walk them forwards.

Tiny example: the ping-pong run

```
time ->    0 ms           10 ms            20 ms
A : [A#1 start]--[A#2 send ping]-------------[A#3 recv pong]
                       \                          ^
                        \ ping                   / pong
                         v                       /
B : [B#1 start]---------[B#2 recv ping]--[B#3 send pong]
```

Its edges: A#1 to A#2 (PROGRAM_ORDER), A#2 to B#2 (MESSAGE), B#1 to B#2 (PROGRAM_ORDER), B#2 to B#3 (PROGRAM_ORDER and STEP_EFFECT), B#3 to A#3 (MESSAGE), A#2 to A#3 (PROGRAM_ORDER). A lane per node, time running left to right, messages as slanting arrows: this is called a **space-time diagram**, and it is the main picture CHRONOS draws.

Technical view

- **Vertices:** events, identified by their log position `seq`, which gives dense integers from 0 to n-1.
- **Edges:** taken from each event's `parents` field, each with its edge kind (Section 12).
- **Acyclic by construction:** every parent has a smaller `seq` than its child, so no edge can point forward and no cycle can exist. The log order is already a topological order, for free.
- **Storage:** compact primitive arrays: for each event a short list of parent positions and kinds, plus a children index built on demand. A run has about three edges per event; we expect a million events to fit comfortably in memory, and Phase 16 measures it.

- **Two ways to build it:** incrementally while recording (the graph subscribes to the recorder) or in one pass over a stored log. Both use the same `append(event)` logic.

Queries the graph must answer

Query	Meaning	Method
happens-before(a, b)	Could a have influenced b?	Compare vector clocks; cost proportional to the node count
concurrent(a, b)	Neither could influence the other	Both happens-before checks fail
causal past(e)	All suspects for e	Walk parents backwards from e
causal future(e)	Everything e could have influenced	Walk the children index forwards from e
path(a, b)	A chain explaining how a led to b	Shortest path inside the past of b, with certain and OMISSION edges cheaper than other potential ones
node lanes	Each node's events in order	Per-node index
consistent cuts (Stretch)	Snapshots of the whole system at points that are not log prefixes	Frontier computation from vector clocks

Built-in self-check

Happens-before can be answered in two independent ways: vector clocks and graph reachability. CHRONOS can run both and compare. Any disagreement means a bug in the clocks, the graph, or the recorder. This one cheap check guards three components at once (FR-15).

Own implementation versus a graph library

Graph libraries and graph databases exist. We do not need them: our graph is a plain DAG with dense integer ids that only grows at the end, which arrays handle better than a general-purpose structure, and implementing it is a stated learning goal.
Recommendation: implement it ourselves (D-14 in Appendix B).

Presentation

The graph exports to an ASCII space-time diagram first (available from Phase 6), then to DOT text that Graphviz can draw, then to JSON and to the static HTML report (Phase 15a, D-18).

Inputs and outputs

Consumes: events from the recorder or the store. **Produces:** query answers and exports. **Used by:** time-travel session, comparator, counterfactual intervention search, visualiser. **Depends on:** the event model and the clock comparison function. **Tested by:** the worked examples of Section 11 and 12, the vector-versus-reachability self-check on thousands of randomly generated runs, and acyclicity checks.

14. State and snapshot model

CHRONOS rebuilds the state of any node at any past moment by reading the log, and saves full-world checkpoints at intervals so that a runnable world can be restored quickly when we want to continue execution from the past.

Plain idea

A computer game has two kinds of saves. A *save file* freezes the whole game right now; you can load it and keep playing. A *diary* of everything you did lets you work out where you were at any earlier moment, but you cannot play from the diary directly. CHRONOS keeps both: the event log is the diary, and checkpoints are the save files.

Tiny example from RE

Node `p` holds: role = PRIMARY, data = {x:1, y:2}, heartbeatCounter = 12. After it handles write W7 its state records data = {x:1, y:2, z:7}. The log stores that new state with the step's trigger event. To know `p`'s state at event 4521, find the latest state record for `p` at or before position 4521. No application code runs.

What state is

- **Node state:** everything one node remembers, held in a single serialisable object with a volatile zone (lost on a crash) and a durable zone (survives a crash) (D-04).

- **World state:** all node states, plus what is *between* nodes: in-flight messages, pending timers, each node's clocks and per-node event counter, each random stream's position, the virtual time, active partitions and fault-rule progress (including per-sender message ordinals, D-13), the last-writer map of every state field (D-17), the environment's own clock values, and the agenda.

The state contract (D-04)

For snapshots and digests to work, a node's state must be:

1. **All of its memory:** nothing important lives in static fields or hidden objects.
2. **Plain data:** numbers, strings, booleans, enums, lists, and maps with a defined order; no references to threads, the runtime or the context.
3. **Canonically encodable:** the same state always produces the same bytes (fixed field order, sorted map keys). *Why:* equal states must have equal digests, or comparison would report false differences.

Zones, codec and last writers (D-04, D-17)

Two zones. The state object has a **volatile zone** and a **durable zone**. The volatile zone is lost when the node crashes. The durable zone survives a crash and stands for the node's disk. A change to the durable zone commits at the end of the step, atomically with the step's events, so a crash can never separate a durable write from the event that explains it. Until Phase 12b a node has only a volatile zone and crash-stop is the only crash model. The stored format carries a zone tag from Phase 8, so no migration is needed later.

State codec. Each node declares a state codec through the Node API: its state type plus a canonical encode and decode. The runtime never guesses how to serialise a node's memory.

Last writers. After every step the engine compares the digest of each top-level state field with its previous digest. Per node and per field it records the **last-writer event**: the trigger event of the step that last changed that field. Field names are zone-qualified (for example `volatile.role`, `durable.log`). A field that was never written has the node's start event as its last writer, and after a restart the restart event takes over for the fields it resets. Invariants read node state through a recording accessor, so the fields they read are known exactly. The last writers of those fields are the **witnesses** of a violation, and each becomes a WITNESS edge into the INVARIANT_VIOLATED event. In v1 invariants may read node state fields only. The map is part of world state and of every checkpoint.

Consistent cuts and why CHRONOS gets them for free

Plain idea: a group photo is consistent if nobody appears holding a letter that nobody has yet written. *Technical:* a **cut** is a set of events that includes every predecessor of each member; it is **consistent** if for every receive in the cut, the matching send is also in the cut. Every prefix of the CHRONOS log is a consistent cut, because parents always precede children (Section 10).

In real distributed systems nobody can freeze everything at once, so algorithms such as Chandy and Lamport's snapshot algorithm use marker messages to assemble a consistent picture. CHRONOS does not need that: the runtime owns the only thread and stops between steps, so it can copy the whole world at an exact instant. That is a direct benefit of the controlled execution model.

Event sourcing

Plain idea: a bank statement lists every deposit and withdrawal; the balance is derived. *Technical:* store the sequence of changes as the source of truth and derive current state from it. CHRONOS is an event-sourced system: the log is primary, and node states, graphs and timelines are derived views.

Three ways to store state history

Option	What the log contains	Advantages	Disadvantages
A. Full post-state on change	The complete node state after each step that changed it	Simplest; reconstruction needs no application code; trivial to inspect and diff	Log grows with state size times number of changes
B. Deltas	Only the difference from the previous state	Compact	Needs diff and patch logic; a bug corrupts history
C. Digests plus periodic snapshots	A digest every step, full snapshots every K steps	Smallest log	State between snapshots needs re-execution of code; cannot inspect without running

Recommendation: A (D-10), with a digest always recorded. Demo applications keep node states small (kilobytes). The storage layer exposes a *state record policy* so we can move to B or C later without changing the model, but only if Phase 16 measurements show a need.

Reconstruction

- **Node state at position p:** each node has an index of (position, state record); a binary search finds the latest record at or before p.
- **In-flight messages at p:** messages whose send is at or before p and which have no delivery or drop at or before p; starting from the nearest checkpoint avoids scanning from the beginning.
- **Pending timers at p:** the same idea with timer set, fired and cancelled events.
- **Clocks and virtual time at p:** taken from the last event of each node at or before p.
- A **world view** at p combines all of these, plus the last-writer map (D-17). It is read-only and needs no application code.

Checkpoints

A checkpoint is a complete, runnable image of the world at a step boundary: everything listed under *World state* above. It is written every K steps (default 1,000), at every branch point, and at the end of the run. Restoring one builds a live world that can continue stepping. Checkpoints are what *branching* and *resumed replay* need; reading a past state alone does not need them but uses them to start folding closer to the target.

State digests

A digest is a short fingerprint of the canonical state bytes. From Phase 8 the runtime also keeps one digest per top-level state field, which is how it detects which step last changed which field (D-17). It lets replay verification and timeline comparison check equality cheaply, and it detects corruption. We start with the JDK's SHA-256 for simplicity and reconsider a faster non-cryptographic hash only if Phase 16 measurements show hashing is a bottleneck.

Inputs and outputs

Consumes: events with state records, checkpoint files. **Produces:** node states, in-flight sets, world views, runnable worlds. **Used by:** time-travel session, replay engine, branching, comparator. **Tested by:** *reconstruct equals live* (state rebuilt from the log matches state

seen during the run, at every step); *round trip* (snapshot, restore, continue gives the same events as an uninterrupted run); digest stability across two separate JVM launches; canonical-encoding tests for map ordering.

15. Replay model

Deterministic replay re-executes the application under the same recorded decisions and checks, event by event, that it produces exactly the same history. If it does, the recording is trustworthy; if it does not, CHRONOS says exactly where and why it stopped matching.

Plain idea

To replay a card game, you do not shuffle again. You deal the cards in the recorded order. Anyone following the same rules with the same cards must end up with the same game. CHRONOS keeps the cards (the recorded decisions) and checks that the players (the nodes) play exactly as before.

Tiny example

RE runs with seed 42. During recording, the network gave heartbeat #12 a latency of 37 ms. On replay, the decision source returns 37 ms for that same message, `b` reacts as before, and every event matches. Now imagine `b` used the system clock to choose a timeout. On replay the clock differs, `b` behaves differently, and CHRONOS stops with: *mismatch at b#31: expected a timer due at 412 ms, got 409 ms*. The bug is in the application, and CHRONOS pointed at the first event that revealed it.

The determinism equation

An execution is a function of four things: **application code, initial configuration, external inputs, and the decisions** the environment made (delays, ordering, drops, random values, time). Fix all four and the execution is identical. CHRONOS stores the configuration and seed in the timeline header, stores inputs and decisions inside the events, and identifies the code by a version fingerprint.

Sources of nondeterminism and how each is controlled

Source	How CHRONOS controls it
Message latency, loss, duplication	Decided by the decision source; stored in the send and fault events
Which ready item runs next	Stable tie-breaking; the order of trigger events in the log is the recorded choice
Time	Virtual clock only
Random numbers	Identity-keyed hashing and per-node streams; draws are recorded
Faults	Fault plan plus seed; every firing is an event
Client workload	Scripted by the scenario, seeded
Application misbehaviour (hash-order dependence, hidden state)	Determinism contract (Section 9) and this section's verifier
JVM internals (garbage collection, JIT, thread timing)	Irrelevant: nodes are single-threaded and never read them

Kinds of replay

Kind	Purpose	Output
R1. Verification replay from the start	Prove that the log is reproducible	Verdict plus first mismatch, if any
R2. Resumed replay from a checkpoint	The same check, starting mid-run and therefore faster	Verdict for the remaining events
R3. Stepped replay with breakpoints	Let a developer run to an event, a virtual time, a node action or a violation and then stop	A live world paused at the breakpoint (built in Phase 10b; Phase 9 builds R1, R2 and R4)
R4. Prefix replay for branching	Rebuild a live world at a branch point (Section 17)	A world ready to diverge

Kind	Purpose	Output
R5. Node-local replay (Stretch)	Because a node is a function of its input sequence, feed one node its recorded inputs alone and check its outputs	A fast single-node debugging mode

The replay algorithm

1. Read the header: scenario name, seed, configuration, format version, code fingerprint.
2. Build the world, either from scratch or from the nearest checkpoint before the start point.
3. Install the **replay decision source**, which reads answers from the recorded events.
4. Run the ordinary engine loop. Each time it needs a choice (which item next, which latency, which random value, whether a fault fires), the decision source supplies the recorded answer.
5. For every event the engine produces, compare it with the recorded event: id, kind, node, payload digest, parent ids, virtual time, and state digest.
6. On the first mismatch, stop and raise a **determinism violation** with a report: the expected and actual event, a field-by-field difference, the last matching event, and likely causes (a payload difference suggests application logic; a timing difference suggests hidden time use).
7. If every event matches and the log ends, report success with the number of events verified and the log hash.

Code versions

Replay is only meaningful if the application code is the same as when the run was recorded. The header stores the scenario name, a declared version string, and a best-effort hash of the application's compiled classes. A mismatch produces a warning rather than a refusal, because developers legitimately want to replay an old failing log against *fixed* code and see where behaviour now differs. That is itself a comparison (Section 20).

Speed

Because time is virtual, replay runs as fast as the processor allows. An optional pacing mode replays at human speed for demonstrations.

What replay is not

It is not replay of arbitrary Java programs, of thread interleavings inside a node, of external side effects, or a guarantee across code versions. The controlled model (D-01) is what makes the guarantee possible, and the contract is what keeps it true.

A deliberate bad citizen

The demo module will include one intentionally nondeterministic node that violates the contract. Acceptance demonstration AD-2 uses it to prove that the verifier really catches hidden nondeterminism instead of merely passing everything.

Inputs and outputs

Consumes: a timeline (header, events, checkpoints), the application code. **Produces:** a verdict, a mismatch report, or a live world at a breakpoint. **Depends on:** engine, decision source, store, state manager. **Used by:** time-travel session, branching, counterfactual engine, the determinism-check command. **Tested by:** *run twice and compare hashes* (same JVM and two separate JVMs); *replay equals record* over many seeds; *log mutation test* (alter one recorded event; the verifier must fail at exactly that event); *replay from every checkpoint equals replay from the start*; the bad-citizen node must be flagged.

16. Time-travel model

Time travel is a debugging session in which a cursor moves freely over a recorded timeline and every question is answered from the log; only *resuming execution* needs a live world, and resuming always creates a branch.

Plain idea

Think of a video player for an execution. You can pause, scrub, step one frame at a time, jump to a scene, and also ask *why is this on screen?* You cannot edit the film. If you want a different ending, you film a new take: that is a branch (Section 17).

Tiny example from RE

The developer types `goto violation` and the cursor lands on event 4521. `state p` and `state b` both show role = PRIMARY. `inflight` shows one heartbeat message still travelling. `back 3` moves three events earlier. `why 4521` prints the chain of causes that led here. Not one line of application code was executed.

Principles

1. **The past is immutable.** A recorded timeline is never changed (NFR-04). Any change creates a new timeline.
2. **Looking is cheap; acting is a branch.** Inspection reads the log and its indexes. Only continuing execution needs a live world.
3. **Backward is just a cursor move.** Ordinary debuggers cannot run backwards because they would have to undo side effects. CHRONOS never un-runs anything: it has stored the history, so going back means looking at an earlier position.
4. **Positions and step boundaries.** The cursor can rest on any event. Node states shown at a position inside a step reflect the last *completed* step. Resuming execution rounds the position to the end of the current step, because a world can only be restored between steps.

Operations

Command	Meaning	Built on
`goto` position or event id	Move the cursor	Index lookup
`step`, `back [n]`	Move one or n events forward or backward	Cursor arithmetic
`next-on` node, `prev-on` node	Move to the next or previous event of one node	Per-node index
`find` predicate	Search forward or backward by kind, node, message type, payload text, or violation	Indexes plus scan
`state` node	A node's state at the cursor	State reconstruction
`world`	All nodes, in-flight messages, timers, clocks	World view
`inflight`, `timers`	Messages and timers pending at the cursor	World view

Command	Meaning	Built on
`why` event	The causal past (the suspects) and the shortest path of certain causes and OMISSION links (with no baseline, it follows these back to their sources: faults, inputs and starts)	Causal graph
`effects` event	Everything the event could have influenced	Causal graph
`concurrent-with` event	Events with no causal relation to it	Vector clocks
`watch` node.field (Should)	The first or last event where a state field changed or met a condition	Scan of state records
`diff` timeline at cursor	Compare the world at this position with another timeline	Comparator
`run-to` predicate	Resume live execution until a condition, then pause	Replay engine, stepped mode
`branch`	Go live from here, creating a new timeline	Timeline manager

Build order (Section 24). Phase 10a delivers `goto`, `step`, `back`, `state`, `world`, `inflight`, `timers` and `why`, which is enough for AD-3. Phase 11 adds `branch`. Phase 10b adds `next-on`, `prev-on`, `find`, `effects`, `concurrent-with`, `watch`, `diff`, `run-to` and breakpoints; `diff` uses the comparator of Phase 14a.

Views

- **Global view:** every node's state, in-flight messages, timers, and clocks at the cursor.
- **Node view:** one node's history and state changes.
- **Message view:** a message's send, delay, drop or delivery.
- **Causal cone view:** the suspects of one event, drawn as a graph.

The CLI prints these as text first. The static HTML report (Phase 15a) draws them; a live viewer is Stretch (D-18).

Several timelines at once

A session can hold more than one timeline. Switching from the failing timeline to a counterfactual one keeps the cursor aligned by event identity, so the developer can look at the same moment in two parallel universes and see which node states differ.

Going live creates a branch

The first command that would change anything (inject a fault, edit a delivery, change configuration) automatically opens a branch at the current step boundary, labels it, and applies all further commands to that branch. The original timeline is untouched.

A knowledge view (Could)

At any event, the **causal past** is exactly what that node could have known when it acted. A *knowledge view* shows the world as one node could have seen it. For RE, it would show that `b` decided to promote itself having seen nothing from `p` since heartbeat #20. This view explains decisions, not just outcomes, and it costs little once the causal graph exists.

Inputs and outputs

Consumes: a stored timeline, its indexes and checkpoints. **Produces:** textual or visual views, causal answers, and branch requests. **Depends on:** store, state manager, causal graph, replay engine, timeline manager. **Tested by:** *reversibility* (going forward n then back n returns an identical view); *fidelity* (the view at any position equals what a probe observed live at the same position during recording); a file-hash check proving sessions never modify a timeline; seek-latency benchmarks (Section 26).

17. Timeline branching model

A timeline is one immutable recorded execution; a branch is a new timeline that shares its parent's history up to a chosen step and then goes its own way. Timelines form a tree, like commits in Git.

Plain idea

In a video game you can save before a hard fight, try one strategy, then reload and try another. Each attempt is a separate play-through, but they all share everything before the save point. CHRONOS does this for whole distributed executions, and keeps every attempt so they can be compared.

Tiny example from RE

```
T0    baseline, seed 42 ------------------------------> VIOLATION at seq
4521
 |
 |-- T0.1   branch at seq 1200: heal the partition 200 ms earlier ---> no
violation
 |
 '-- T0.2   branch at seq 1180: drop heartbeat #14 ---------------> VIOLATION
at seq 3902
      '-- T0.2.1   branch at seq 2000: crash b at 2100 ms ------> ...
```

All three branches reuse events 0 to 1180 of `T0` without copying them.

What a timeline holds

Item	Meaning
`timelineId`, `label`	Hierarchical id (such as `T0.2.1`) and an optional human label
`parentId`, `branchSeq`	Which timeline it branched from, and the last position that belongs to the parent's prefix; none for a root timeline
Scenario, parameters, seed, fault plan	Everything needed to recreate the run
Interventions	The list of changes applied at the branch point (Section 19)
Node list, format version, code fingerprint	Needed for replay and compatibility
Status	RECORDING, COMPLETED, VIOLATED, TIMED_OUT, or ABORTED
Event log, checkpoints, indexes	The recorded data itself (Section 23 shows the file layout)
Summary	Event count, final virtual time, end-state digest, first violation if any

Rules

1. A completed timeline is immutable.
2. A branch point is a **step boundary**, because a world can only be restored between steps.
3. **The prefix is shared by reference.** Reading a branch means reading the parent's events up to the branch point and then the branch's own events.
4. Position numbering continues: the branch's first own event is the one after `branchSeq`. Shared-prefix events keep their ids and positions.
5. A checkpoint is stored at every branch point, so branching from a branch is fast.
6. The interventions are stored both in the timeline's metadata and as an INTERVENTION_APPLIED event on env at the branch point (id iv:<interventionId>, D-25), and every event the intervention changes carries an `origin` field, so the timeline explains itself.

Branch creation algorithm

1. Choose the parent timeline and the branch point *p*, rounded to a step boundary.
2. Obtain a live world at *p*: restore the nearest checkpoint at or before *p*, then run a prefix replay (R4 in Section 15), verifying events as it goes, until *p* is reached.
3. Create the new timeline record with a new id, the parent pointer, and `branchSeq = p`.
4. Install the **branch decision source**: the same scheduling policy, the same seed and the same identity-keyed randomness as the parent, so unaffected parts of the system behave identically.
5. Record an INTERVENTION_APPLIED event on env for each intervention (D-25), then apply the interventions at *p*.
6. Run until the horizon, an invariant violation, or an empty agenda. The recorder appends new events with positions continuing after *p*.
7. Write the final checkpoint and status.

Three ways to store a branch

Option	Advantages	Disadvantages
Full copy of the prefix	Each timeline is standalone	Disk grows with prefix length times branch count; branching slows with long prefixes
Reference to the parent prefix	Cheap and fast branching; makes sharing explicit and verifiable	A branch depends on its ancestors; ancestors cannot be deleted while children exist
Structural sharing in a specialised persistent structure	Elegant	Far more machinery than we need

Recommendation: reference the parent prefix (D-12). A timeline resolves its ancestor chain once when opened into a list of (timeline, position range) segments, so reading stays simple. The timeline manager (C11, in the analysis layer) owns the id policy and branch creation; the store owns catalogue persistence and segment resolution (D-11).

Operations

Create a branch; list the timeline tree; show a timeline's summary; label or annotate a timeline; delete a *leaf* timeline; export a timeline; compare two timelines (Section 20). **There is no merge.** There are no sound semantics for merging two executions, and the project does not need one.

Branching in parallel

Branches are independent. Each runs in its own isolated world and writes its own files, and the parent data they all read is immutable, so they can run on different threads safely (Section 19). Timeline ids are allocated by the timeline manager before the runs start, so naming is deterministic even with parallel branching.

Inputs and outputs

Consumes: a parent timeline, a branch point, interventions. **Produces:** a new timeline (log segment, checkpoints, metadata). **Depends on:** store, state manager (checkpoints), replay engine (prefix replay), engine, decision source. **Used by:** counterfactual engine, time-travel session, comparator. **Tested by:** *prefix identity* (positions up to `branchSeq` are byte-identical to the parent's); a **no-op branch** (no interventions) reproduces the parent exactly (event-identical, checked directly in Phase 11 and, from Phase 14a, by the comparator reporting no divergence); branch-of-branch consistency; deletion rules; a check that parent files never change.

18. Fault-injection model

A fault is a controlled failure of the environment, applied only by the runtime's network and scheduler, decided deterministically, and recorded as an event; so every faulty run is repeatable and every effect of a fault can be traced back to it.

Plain idea

Picture a chaos laboratory for a postal service. The lab can lose a letter, delay it, deliver it twice, close the road between two towns, or make a clerk faint. Real-world chaos tests are one-offs: the failure happens, nobody can repeat it. In CHRONOS the chaos is a script. It runs the same way every time, and the log records exactly when and why each thing went wrong.

Tiny example from RE

A partition cuts `p` from `b` between 300 ms and 700 ms. While it is active, every message between them is discarded, and each discard is recorded as a MESSAGE_DROPPED event whose parent is the partition's FAULT_INJECTED event. `b` hears no heartbeat, its 300 ms watchdog timer fires with an OMISSION edge from the partition event (D-23), and `b` promotes itself. Asking *why did b promote?* now yields a chain that starts at the partition event.

Fault catalogue

Fault	Plain meaning	Effect in the runtime	Recorded as	Priority
Message drop	A letter is lost	The delivery is removed from the agenda	MESSAGE_DROPPED on `env`	Must
Message delay	A letter is late	Due time increases	FAULT event; the delivery gets a FAULT parent	Must
Network partition and heal	The road between two groups is closed, then reopened	Messages across groups are dropped while active	FAULT_INJECTED, FAULT_CLEARED, one drop per message	Must
Node crash	A node dies silently	Its deliveries are dropped, its timers discarded	NODE_CRASHED	Must
Node restart	A crashed node comes back	Volatile state lost, durable zone kept	NODE_RESTARTED	Should
Message duplicate	A letter arrives twice	A second delivery is created	MESSAGE_DUPLICATED	Should
Message reorder	Two letters swap places on a link	Delays adjusted so the order flips	FAULT event plus delivery parents	Should
Node pause	A node freezes then resumes (like a long garbage-collection pause)	Its items are held until resume	NODE_PAUSED, NODE_RESUMED	Should

Fault	Plain meaning	Effect in the runtime	Recorded as	Priority
Clock skew	A node's clock runs fast, slow or offset	The time the node reads changes; logical clocks do not	FAULT event	Should
Slow link	One link has persistently higher latency	Latency multiplier	FAULT event	Could

Not planned: message corruption and Byzantine behaviour (Section 5).

Build order (Section 24). Phase 12a implements only the partition-and-heal fault, triggered at a virtual time, with its fault events, causal edges and OMISSION edges: enough to reproduce the RE split brain. Phase 12b adds drop, delay, crash, restart, duplicate, reorder, pause, clock skew, slow link, the other triggers, the durable zone and campaigns.

Fault plans

A **fault plan** is a list of rules. Each rule has an id, a **trigger**, an **action** from the catalogue, and optionally a duration, or a maximum number of firings (only for rules triggered by time or by a named event; see the keying rule below).

Trigger	Meaning	Example
At time	At a virtual time	Partition at 300 ms
After event	Immediately after a named event	Crash `b` right after `p#40`
On message id	When one named message is sent	Drop message `p#14`
On nth message	When the _n_th message of one type from one named sender to one named receiver is sent, counted in the sender's own history	Drop the 3rd heartbeat from `p` to `b`
Probabilistic	Each eligible message independently, with probability *p*, decided by a hash of (seed, message id, rule id)	Drop 5% of heartbeats

Trigger	Meaning	Example
On state (Stretch)	When a node's state satisfies a predicate	Crash `p` when its role is PRIMARY

A plan is declared in the scenario, or generated from a seed by a chaos generator.

Keying rule (D-13). Every rule is keyed by time or by identity, never by a global counter, so a branch cannot change a rule's behaviour for unrelated nodes. The *nth message* trigger counts messages in the sender's own history, and that per-sender ordinal is part of world state, checkpointed with the sender. A maximum number of firings is allowed only on rules triggered by time or by a named event. A wildcard sender is allowed only if the rule is flagged **nonLocal**; such a rule voids the locality law M8 for that run, and the run's report says so. Phase 12a implements only the *at time* trigger; the others arrive in Phase 12b.

How faults stay deterministic and replayable

- **Probabilistic triggers use identity-keyed hashing:** whether message `p#17` is dropped depends on a hash of (seed, message id, rule id), never on a shared random stream. The same message meets the same fate in every timeline, which keeps counterfactuals clean (Section 19).
- **Outcomes are recorded in events,** so replay reads them back rather than recomputing.
- **Rule progress** (such as how many matching messages have been seen) is part of world state, so checkpoints capture it.
- **Faults are applied only by the network and scheduler, never inside node code.** A node cannot tell a fault from reality except through its consequences, such as silence, exactly as in a real system.

Faults in the causal record

A fault event is connected to the events it changes by FAULT edges. A delayed delivery has the fault as a parent; a drop has the fault and the send as parents; a crash event has the fault as its parent. So the question *why was this message dropped?* is answered by a graph query rather than by guesswork. Faults also cause things that never happen: when a partition, crash or pause silences a node's inputs, the watchdog or timeout that fires in response is linked back to the fault by an OMISSION edge (D-23, Section 12). Phase 12a

covers partition membership; crash and pause scopes arrive in Phase 12b, where a fault touches a node if the node has received messages from the crashed or paused node before.

Faults during a session

An ad-hoc fault typed during a debugging session is an *intervention*: it opens a branch, and the fault is recorded there (Sections 16 and 19). The original timeline never changes.

Chaos campaigns

A **campaign** runs many seeds, or many randomly generated plans and lists the timelines that ended in an invariant violation. This is how we *find* a failing baseline to debug. In Phase 12b it runs seeds one after another; from Phase 13b it reuses the parallel runner of the counterfactual engine.

Fault coverage

Every run reports which rules fired and how often. A rule that never fired is flagged, because a fault plan that never triggers is a test that silently tested nothing.

Known simplification

A node crashes *between* steps, never halfway through a handler. Real crashes can leave partial effects. Modelling that needs write-ahead semantics inside nodes and is deliberately out of v1. Scenario authors can approximate it by splitting a handler's work across messages or timers.

Inputs and outputs

Consumes: a fault plan, interventions, the decision source, the agenda. **Produces:** fault events with causal links, and altered agenda items. **Depends on:** engine, network simulator, event model. **Used by:** counterfactual engine, campaigns, time-travel session. **Tested by:** a partition drops exactly the cross-group messages and heal restores delivery; identical plan and seed give identical logs; a probabilistic rule hits the same message in a parent and in its branch; coverage reports flag unfired rules.

19. Counterfactual execution model

A counterfactual run answers *what if?*: it branches a recorded timeline at a chosen step, applies one deliberate change, runs forward, and reports whether the failure still happens. This turns a suspicion into evidence.

Plain idea

If I had left home five minutes earlier, would I have avoided the accident? You cannot answer by crossing the accident out of the story, because everything after the changed moment would have unfolded differently: other cars, other lights. You have to live the day again from that moment with the one change. A counterfactual run does exactly that, in the simulation.

Tiny example from RE

The suspect is the partition at 300 ms. The developer branches the failing timeline `T0` just before the partition and removes it. The run continues: heartbeats arrive, `b` never promotes itself, and the single-primary invariant holds. Verdict: **the failure disappears**. In this world, the partition as timed was a *but-for cause* of the failure. Had the failure persisted, the partition would have been cleared and the search would continue elsewhere.

What a counterfactual query is

A query is a combination of: a **baseline timeline** T, a **branch point** p, a set of **interventions** I, a **horizon** H, and a **question**. The result is a new timeline T′ plus a report. Typical questions:

- **Necessity (but-for):** if this event is removed or altered, does the failure disappear?
- **Sufficiency:** if this event is added to a healthy timeline, does the failure appear?
- **Sensitivity:** how far can the change shrink and still cause the failure (for example, what is the shortest partition that still breaks RE)? Answered by running a sweep of related queries.

Intervention catalogue

Intervention	Example	How it is applied	Kind
Drop a message	Drop heartbeat `p#14`	The delivery is marked dropped, caused by the intervention	Realistic
Delay a message	Delay `p#14` by 250 ms	Due time is increased	Realistic
Reorder deliveries	Swap two messages on a link	Delays are adjusted so order flips	Realistic
Duplicate a message	Deliver `p#14` twice	A second delivery is created	Realistic
Crash, restart, or pause a node	Crash `b` at 2100 ms	A fault action at the branch point	Realistic
Create, move or remove a partition	Heal 200 ms earlier	A fault action with new times	Realistic
Change a node's clock skew	Add 80 ms to `b`'s clock	A fault action	Realistic
Add or remove a client input	Add write W8	An agenda item is added or removed	Realistic
Change configuration from p onward	Heartbeat timeout 300 ms to 500 ms	Configuration patch applied at p	Realistic
Change a random stream	Reseed one node's stream	Stream replaced at p	Realistic
Patch node state (Could)	Set `b.role` to BACKUP	State edit at p	**Synthetic**

Changing configuration from the *start* of a run is not a branch; it is a new root timeline (a *variant run*). **Synthetic** interventions may produce situations the real system could never reach, so a timeline created with one is permanently marked `synthetic` and reports say so.

Marking and build order (Section 24). Every intervention is recorded as an INTERVENTION_APPLIED event on `env` at the branch point (D-25). Phase 13a implements exactly one intervention type, removing the partition or changing its heal time, as a single

run with a verdict and no worker pool. Phase 13b adds the rest of the catalogue and the parallel pool.

Three subtleties that make counterfactuals hard

1. The future must be recomputed, not edited. After p, what happens depends on everything before it. Deleting one event from the old log would leave a history that no execution could have produced. CHRONOS therefore re-runs the engine from p.

2. Butterfly noise. If one extra random draw shifted every later random value, the new timeline would differ from the old one everywhere, for reasons unrelated to the intervention. Four design rules prevent this: identity-keyed randomness (Section 9), per-node random streams, stable tie-breaking by (due time, node index, kind, message id) instead of insertion order, and identity-keyed ids for environment events (D-24). They give the **locality property**:

> A node that has no causal connection to the intervention behaves identically in the new timeline.

The argument is short. An unaffected node receives messages from unaffected senders; those senders produce identical messages by induction, with identical identity-keyed delays, so the arrival times and the tie-breaking keys match and the node acts identically. The property holds for fault plans with no nonLocal rules (D-13). It is also one of our most valuable tests: a violation means either hidden nondeterminism or a flaw in the runtime design.

3. Continuation policy. After p, who decides what happens?

Option	Idea	Advantages	Disadvantages
Continue	Same scheduling policy, seed and identity-keyed decisions as the parent	Simple; unaffected messages automatically repeat their original timings	Cannot force a specific original order for affected parts
Pinned	Follow the parent's original order wherever it still makes sense	Closer to the old story for affected parts	Complex matching rules; little extra benefit, since Continue already reproduces unaffected timings

Recommendation: Continue (D-15). Pinned is a Stretch idea, built only if the first needs it.

Verdicts

Each counterfactual report states one verdict:

- **FAILURE_DISAPPEARS:** the run completed with no violation.
- **FAILURE_PERSISTS:** the same invariant was violated again.
- **DIFFERENT_FAILURE:** a different invariant was violated, or the violation has a different causal origin.
- **NO_EFFECT:** the new timeline is identical to the baseline; the intervention changed nothing (for instance, the message was going to be dropped anyway). This is itself informative.
- **INCONCLUSIVE:** the horizon ended before the failure could have happened.

The report also includes the new timeline id, the divergence summary produced by the comparator (Section 20), and the set of events affected by the intervention (its causal future). Phase 13a reports FAILURE_DISAPPEARS, FAILURE_PERSISTS, NO_EFFECT and INCONCLUSIVE, decides NO_EFFECT by direct event comparison, and has no divergence summary until the comparator exists (Phase 14a). DIFFERENT_FAILURE needs the comparator's causal-origin test and arrives in Phase 13b.

Searching for causes automatically

- **Seed campaign (Should):** run seeds 1 to N, each with a chaos plan, in parallel, and list the timelines that violated an invariant. This produces failing baselines to debug.

- **Intervention search (Stretch):** given a violation V, take its causal past, keep only the events that can be changed (messages, timer firings, client inputs, fault events), and run one counterfactual per suspect that removes or delays it. Suspects whose removal makes the failure disappear are **critical events**, ranked by causal distance to V. The causal-past slice keeps this tractable: a few hundred runs, not millions.

Running many counterfactuals in parallel

Every counterfactual is independent: it builds its own world from a checkpoint, uses its own recorder and files, and only reads the immutable parent timeline. Runs go to a fixed-size pool of worker threads (Java executors). Results are collected and sorted by suspect id, so the report is identical whatever order the threads finish in and however many threads are used. This is the main place where real Java concurrency is exercised in CHRONOS, and it is safe *because* of the isolation.

Honest limits

- A counterfactual shows what happens in the **simulated world under CHRONOS's rules**. It is evidence about the model, not proof about reality. The rule *same seed and identity-keyed randomness* is what defines the closest alternative world; other reasonable rules might give different answers.
- **But-for tests can miss overdetermined failures.** If two independent partitions would each cause split brain, removing either one alone leaves the failure in place. CHRONOS reports FAILURE_PERSISTS, and the developer must combine interventions. Teaching note: but-for causality is the standard starting definition and has known gaps like this.
- Synthetic interventions are flagged as such, always.

Inputs and outputs

Consumes: baseline timeline, branch point, interventions, horizon. **Produces:** new timelines and counterfactual reports. **Depends on:** timeline manager, replay engine (prefix replay), engine, fault injector, comparator, invariant checker, causal graph (for suspects). **Tested by:** known-answer scenarios with a planted cause (the engine must confirm it and clear the innocent suspects); the locality property on random interventions; a no-op intervention giving NO_EFFECT; reports identical across thread counts.

20. Timeline comparison and first-divergence detection

To compare two timelines, CHRONOS matches their events by identity, finds where each node first stops matching, keeps only the differences that were not themselves caused by an earlier difference (the *causal roots*), and reports the earliest root as the **first causal divergence**, together with the causal path from it to the failure.

Plain idea

Two recordings of the same play differ at the end: costumes, props and lines are all different. Pointing at everything is useless. The useful finding is: *the lead actor missed one cue here, and everything else followed from that.* Comparison in CHRONOS finds that first missed cue.

Tiny example from RE

Compare `T0` (partition from 300 to 700 ms, ends in a violation) with `T0.1` (the partition heals at 500 ms, no violation). The differences that appear are:

- the INTERVENTION_APPLIED event `iv:heal-early` exists only in `T0.1`;
- the partition-heal event happens at 700 ms in `T0` and at 500 ms in `T0.1`;
- `b`'s watchdog firing at 600 ms exists only in `T0`;
- `b`'s promotion to PRIMARY exists only in `T0`;
- the writes that `p` and `b` both accepted exist only in `T0`;
- the INVARIANT_VIOLATED event exists only in `T0`.

Only the first of these has no earlier difference behind it: the changed heal event is its child (D-25), and everything else follows from the heal. It is the root, and it is the intervention's own event (law M10). The root lives only in `T0.1`, so the explanation path is drawn in `T0`, where the failure is, starting from the root's anchor: the last event on `env` that both timelines still share. CHRONOS prints the chain: *partition at 300 ms, heartbeats dropped, watchdog fires (OMISSION link), `b` promotes, two primaries, violation.*

The algorithm

The reference timeline is called A (usually the baseline) and the other B.

1. **Skip the shared prefix.** If B descends from A (or both from a common ancestor), everything up to the branch point is identical by construction (Section 17); only later events are compared.

2. **Align by identity.** Events are matched by `eventId` (events on `env` by their identity keys, D-24). Each matched pair is compared by content digest (Section 10), which includes `origin`. Every event ends up in one class: **Same**, **Changed** (same id, different digest), **Missing** (only in A), **Extra** (only in B), or **Reordered** (same events but in a different local order at one node). Two comparison strengths exist: *structural* (ignores virtual time) and *strict* (includes it).
3. **Find each node's first local divergence.** For every node (including `env`), walk the two local sequences side by side to the first mismatch. A node's behaviour is a function of its input sequence, so a first mismatch is one of two things: an **input difference** (a message, timer or client input that arrived differently or not at all), whose cause lies elsewhere; or an **output difference despite identical inputs**, which a deterministic application cannot produce and is flagged as *suspected application nondeterminism* (FR-43).
4. **Collect the divergent set D:** every event that is not Same.
5. **Find the causal roots inside each timeline.** In timeline A, and separately in timeline B, walk the events in that timeline's own log order over that timeline's own causal graph. Mark an event *tainted* if it belongs to D or has a tainted parent. An event in D is a **root** if none of its parents is tainted. Taint never crosses from one timeline to the other. Each pass is linear in the number of events. Collect the roots of both timelines.
6. **Name the first causal divergence.** Order all roots by (virtual time, node order with `env` last, event id). Lamport and vector values are never compared across timelines, because they belong to different executions. Inside one timeline, a root that happened before another comes first. Report *all* roots with the earliest as primary: two roots can be concurrent, in which case neither is causally first and the order shown is only a display convention.
7. **Draw the divergence cone.** The cone is the causal future of the roots within D: the events whose difference is explained. Any event in D that lies in no root's cone is *unexplained* and flagged, since it should be impossible.
8. **Explain the path.** In the timeline that contains the failure, find the shortest path from the first causal divergence to the first INVARIANT_VIOLATED event. Certain and OMISSION edges cost 1 and other potential edges cost 2, so the path prefers real *because-of* links (Section 12). If the primary root exists only in the other timeline, the path starts at the root's **anchor**: the last Same event on the root's node before the divergence. Each hop is printed as one plain-language line citing event ids. The anchor rule is validated in Phase 14a with KA-1 and a hand-built cross-timeline case.

Why "first" means causally first

The log position of two concurrent events is arbitrary: it is just one valid way to list them. If two unrelated differences happen on different nodes, calling the one with the smaller position "first" would be an accident. Causal rootedness is the meaningful notion, and it is why Section 12 matters so much.

What the report contains

- Counts of Same, Changed, Missing, Extra and Reordered events.
- Each node's first local divergence.
- All roots and the primary first causal divergence.
- The divergence cone, with its size and the affected nodes.
- The explanation path to the failure.
- Invariant outcomes in A and in B.
- Any suspected-nondeterminism and unexplained-divergence flags.

Edge cases

Case	Behaviour
Identical timelines	Reports *no divergence*
One timeline ends earlier	The remaining tail is Missing or Extra; the root is the point where one side stopped
Different seeds	Allowed, with a warning: nearly everything differs from the first random draw
Divergence that later disappears	Reported as an *absorbed divergence*: the states re-converged
Several concurrent roots	All are listed; the earliest is marked primary
Old log compared with a replay against fixed code	Works the same way: the roots show where the code change first mattered
Synthetic intervention in the history	Report is labelled synthetic

Case	Behaviour
Primary root exists only in the other timeline	The path is drawn in the timeline that contains the failure, starting at the root's anchor: the last Same event on the root's node before the divergence (D-16)

Inputs and outputs

Consumes: two timelines (events, causal graphs, invariant results). **Produces:** a divergence report and a path. **Depends on:** store, causal graph, event model. **Used by:** counterfactual engine (every verdict), time-travel `diff`, visualiser, acceptance demonstrations AD-5. **Complexity:** linear in the number of events after indexing. **Tested by:** *identity* (compare a timeline with itself gives nothing); *no-op branch* (compare with a no-intervention branch gives nothing); *known answer* (compare a timeline with a branch made by intervention I and the primary root must be I's own INTERVENTION_APPLIED event (D-25)); *planted perturbation* (copy a log, alter one payload, the root must be exactly that event); randomised runs where the roots must always lie in the causal future of the intervention; a hand-built case with two concurrent roots.

21. Major system components

CHRONOS has eighteen components, C1 to C18. Each entry below says what the component does, why it exists, what goes in, what comes out, and who it talks to. Components talk through interfaces and immutable data (events, timelines, reports), never by reaching into each other's internals. Sections 9 to 20 contain the full model for each idea; this section is the index and the contract.

C1. Node SDK (the Node API)

- **Does:** defines what an application node is: handlers for start, message, timer (and recovery after a crash from Phase 12b), the state codec that declares the node's state type and its volatile and durable zones (D-04), plus the node context through which it sends messages, sets timers, reads virtual time, draws random numbers, reads configuration and annotates the log.
- **Why:** it is the narrow doorway that makes the controlled model possible (D-01).
- **In:** calls from the engine (deliver this message, fire this timer). **Out:** requests to the engine (send, set timer, draw random value), and the node's state object.

- **Talks to:** C2 (engine) only. Application code never sees any other component. C1 lives in its own module, chronos-sdk (D-26).

C2. Simulation Engine

- **Does:** owns the world: agenda, scheduler, virtual clock, simulated network, timer service, node lifecycle. Runs the step cycle of Section 9.
- **Why:** this is where nondeterminism is eliminated and execution becomes recordable and repeatable.
- **In:** scenario, seed, a decision source, a fault plan. **Out:** events to the recorder; a final world; invariant-check calls.
- **Talks to:** C1 (calls nodes), C3 (asks for decisions), C4 (stamps events), C5 (emits events), C8 (state digests, checkpoints), C12 (fault actions), C15 (invariants).

C3. Decision Source

- **Does:** answers every nondeterministic question the engine asks. Record mode answers from a seeded policy, replay mode from the log, branch mode from the parent log and then from the policy plus interventions. The interface and the record mode live in chronos-runtime; the replay and branch modes live in chronos-analysis, because they read stored logs (D-19).
- **Why:** one seam for all nondeterminism, so record, replay and branching share a single engine (D-19).
- **In:** the question plus its identity (message id, node, purpose), the seed, a log reader, interventions. **Out:** the answer.
- **Talks to:** C2 (asked by it), C6, through the LogReader interface of chronos-model (replay and branch modes read recorded events).

C4. Clock Service

- **Does:** keeps per-node Lamport counters and vector clocks, stamps every event, and attaches clock values to messages. Provides the vector comparison function. Treats env like a node and merges clocks along every edge kind, not only messages (Section 11).
- **Why:** logical time is what makes causality exact and comparison possible.
- **In:** local, send and receive hooks. **Out:** a stamp for each event; message clock values; comparison results.
- **Talks to:** C2 (called by it); used for comparison by C7, C10, C14.

C5. Event Recorder

- **Does:** assigns sequence numbers, collects the events of each step as an atomic group, and passes them to one or more sinks through the EventSink interface of chronos-model (in-memory list, the causal graph, the store's writer).
- **Why:** one choke point guarantees that every event is recorded the same way and in order.
- **In:** event groups from C2. **Out:** a stream of immutable events to sinks.
- **Talks to:** C2 (receives), C6 and C7, only through the EventSink interface (the store's asynchronous writer persists; the graph builds live).

C6. Event Store

- **Does:** persists events in an append-only segmented log, writes checkpoints, keeps indexes by position, node, kind and virtual time, persists the timeline catalogue, resolves branch prefixes into segments, and recovers from a torn tail. It implements the EventSink and LogReader interfaces of chronos-model (D-11, D-26).
- **Why:** timelines must outlive the process and be queryable without re-running.
- **In:** events, checkpoint images, timeline metadata. **Out:** event ranges, indexed lookups, checkpoints.
- **Talks to:** C5 (writes), and read by C3, C7, C8, C9, C10, C11, C14.

C7. Causal Graph Engine

- **Does:** builds the causal DAG and answers happens-before, concurrency, causal-past, causal-future and path queries (Section 13).
- **Why:** converts a log into an explanation tool.
- **In:** events with parents (live or from the store). **Out:** query answers, DOT and JSON exports, ASCII space-time diagrams.
- **Talks to:** C5 or C6 (input), used by C10, C13, C14, C16.

C8. State and Snapshot Manager

- **Does:** canonical state encoding, whole-state and per-field digests, the last-writer map (D-17), checkpoint creation and restoration, and read-only state reconstruction (Section 14).
- **Why:** without it nothing can be inspected at a past time, and nothing can be resumed.

- **In:** node state objects, world images, state records. **Out:** digests, checkpoints, reconstructed node states and world views.
- **Talks to:** C2 (digest and checkpoint hooks), C6 (storage), used by C9, C10, C11, C14.

C9. Replay Engine

- **Does:** runs verification, resumed, stepped and prefix replays by installing the replay decision source in the ordinary engine and comparing each regenerated event with the recording (Section 15).
- **Why:** proves the determinism claim and supplies live worlds for time travel and branching.
- **In:** a timeline, a start point, breakpoints. **Out:** a verdict, a mismatch report, or a paused live world.
- **Talks to:** C2, C3, C6, C8; used by C10, C11, C13.

C10. Time-Travel Session

- **Does:** holds the cursor and answers navigation and inspection commands from the log (Section 16).
- **Why:** the developer-facing heart of the debugger.
- **In:** commands, a timeline. **Out:** views, causal answers, and branch requests.
- **Talks to:** C6, C7, C8, C9, C11; presented by C16.

C11. Timeline Manager

- **Does:** the catalogue of timelines: ids, parent links, branch points, metadata, creation of branches, deletion of leaves (Section 17). It owns the timeline id policy and branch creation; C6 persists the catalogue (D-11).
- **Why:** branching needs a single authority for identity and ancestry.
- **In:** branch requests, interventions. **Out:** new timelines and the tree listing.
- **Talks to:** C6, C8, C9, C2, C3; used by C10, C13.

C12. Fault Injector

- **Does:** evaluates fault-plan triggers, applies actions to the agenda, network and nodes, and records fault events with causal links, including OMISSION links to the timers that fired because of silence (Section 18, D-23).
- **Why:** controlled, repeatable failure is half of the CHRONOS value.

- **In:** a fault plan, interventions, the agenda. **Out:** altered agenda items and fault events; a coverage report.
- **Talks to:** C2 (it runs inside the step cycle), C3 (identity-keyed decisions).

C13. Counterfactual Engine

- **Does:** executes counterfactual queries, seed campaigns and (Stretch) intervention searches, in parallel on a worker pool (Section 19).
- **Why:** converts suspicion into evidence.
- **In:** baseline timeline, branch point, interventions, horizon. **Out:** new timelines and verdict reports.
- **Talks to:** C11, C9, C14, C7, C15; presented by C16.

C14. Comparator and Divergence Analyzer

- **Does:** aligns two timelines, classifies differences, finds causal roots separately in each timeline, names the first causal divergence (the earliest root, D-16) and the explanation path (Section 20).
- **Why:** answers where two executions parted and why that matters.
- **In:** two timelines. **Out:** a divergence report.
- **Talks to:** C6, C7, C4 (comparison); used by C10, C13, C16.

C15. Invariant Checker

- **Does:** evaluates global rules after each step using the engine's god's-eye view, and records violations as events. Invariants read node state through a recording accessor, and the last-writer events of the fields they read become the witnesses of a violation (D-17). Per-node assertions are also supported.
- **Why:** a failure must be a precise, checkable thing for causal analysis and counterfactual verdicts to have an anchor.
- **In:** the world after a step, the registered invariants. **Out:** INVARIANT_VIOLATED events with witnesses.
- **Talks to:** C2; read by C13, C14, C10.

C16. Presentation Layer

- **Does:** the command-line interface, text and DOT and JSON exporters, and a static, self-contained HTML report generated by the CLI (D-18). A live local viewer on the JDK's built-in HTTP server is Stretch (post-v1).
- **Why:** people need to see and drive CHRONOS. It holds no logic of its own; it only calls the components above.
- **In:** user commands; reports and views. **Out:** text, files, pages.
- **Talks to:** C10, C11, C13, C14, C7.

C17. Demo Applications and Scenarios

- **Does:** ping-pong, a causal-ordering demo, RE with a seeded bug, and at least two further known-answer bug scenarios; one deliberately nondeterministic node.
- **Why:** a debugger needs something to debug, and known answers are how we test the analyzers.
- **In:** the Node API. **Out:** runnable scenarios with documented planted causes.
- **Talks to:** C1 only, through the chronos-sdk module (D-26).

C18. Test and Benchmark Harness

- **Does:** shared test utilities (scenario runners, log comparators, random-schedule generators), determinism checks, the banned-API scan, and benchmark scenarios (Sections 25 and 26).
- **Why:** every claim in this document needs evidence.
- **In:** all components. **Out:** pass or fail results and measurements.
- **Talks to:** every component, from outside.

22. Data flow through the system

Data moves through CHRONOS in four flows: **record**, **inspect**, **branch**, and **compare**. Every flow either produces or reads an immutable timeline, which is why the flows never interfere with each other.

The main pieces of data

Artifact	Made by	Used by	Can it change?
Scenario (nodes, workload, fault plan, invariants)	C17	C2, C11	Code; recorded by name and parameters
Event	C2 through C4 and C5	C6, C7, C8, C9, C10, C14	Never
Checkpoint	C8	C6, C9, C11	Never
Timeline (header, events, checkpoints, indexes)	C11 and C6	everyone downstream	Never after completion
Intervention	the user or C13	C11, C12, C3	Recorded with the branch
Report (verdict, divergence, replay mismatch)	C13, C14, C9	C16	Generated

Flow A: recording a run

1. The CLI (C16) names a scenario, seed, fault plan and horizon and hands them to the engine (C2).
2. The timeline manager (C11) allocates a timeline id; the store (C6) writes the header.
3. The engine builds the world and installs the **record-mode decision source** (C3). Nodes are started.
4. For each step the scheduler takes the next agenda item. The fault injector (C12) may alter it; the decision source settles any choice.
5. The clock service (C4) stamps the trigger event, and the engine calls the node handler (C1).
6. The node's effects come back through its context. For each one, C4 stamps an event and C3 supplies the latency or random value; the engine puts new items in the agenda.
7. The state manager (C8) canonically encodes the node state and records its digest (and the full state if it changed).
8. The engine hands the step's events to the recorder (C5) as one group; the recorder assigns positions.
9. The recorder fans the events out: the live causal graph (C7), an in-memory sink, and an asynchronous writer into the store (C6).

10. The invariant checker (C15) inspects the world; a violation becomes an INVARIANT_VIOLATED event in the same stream.
11. Every K steps C8 writes a checkpoint through C6.
12. At the end C6 flushes and writes the summary; C11 marks the timeline COMPLETED, VIOLATED, or TIMED_OUT.

Result: an immutable timeline on disk.

Flow B: inspecting and time travelling

1. The CLI opens a session (C10) on a timeline. C11 supplies the metadata; C6 resolves the ancestor segments and loads the indexes; C7 builds the graph from the log.
2. `goto`, `step` and `back` move the cursor using C6's indexes.
3. `state` and `world` ask C8 to reconstruct from state records and the nearest checkpoint.
4. `why` and `effects` ask C7 for the causal past or future and for the shortest path.
5. Results return to C16 as text or pictures.

Nothing in this flow writes to the timeline; sessions are read-only until the user branches.

Flow C: branching and counterfactuals

1. The user (through C16) or the counterfactual engine (C13) submits a query: parent timeline, branch point, interventions, horizon.
2. C11 allocates a child id and writes the metadata.
3. The replay engine (C9) rebuilds a live world at the branch point: C8 restores the nearest checkpoint (read through C6) and C9 replays forward to the point, verifying as it goes.
4. The **branch-mode decision source** (C3) is installed, and the interventions are handed to the fault injector (C12) and the engine, which records one INTERVENTION_APPLIED event per intervention on env (D-25).
5. The engine (C2) runs forward. The recorder and store (C5, C6) append new events after the branch point, and the invariant checker (C15) watches.
6. C13 asks the comparator (C14) to compare the new timeline with its parent, derives the verdict, and returns a report.

For a campaign or an intervention search, C13 runs many of these in parallel, each fully isolated.

Flow D: comparing two timelines

1. The comparator (C14) opens both timelines through C6 and skips their shared prefix.
2. It aligns events by identity, classifies differences, and computes each node's first local divergence.
3. It uses the causal graph (C7) to find the causal roots of each timeline (taint is computed inside each timeline, D-16) and the divergence cone.
4. It asks C7 for the explanation path from the first causal divergence to the failure.
5. The report goes to C16, to the time-travel session (for `diff`), and to the counterfactual engine (for verdicts).

Two rules that keep the flows clean

- **Data flows downhill only.** The engine produces events; analysis components consume them. Analysis components never call back into a running engine, except through the replay engine and timeline manager, which build *new* worlds.
- **Everything shared between threads is immutable.** Events, checkpoints and completed timelines are never modified, which is what makes parallel counterfactuals and background writing safe.

23. Proposed internal architecture

CHRONOS is one Java 21 codebase organised as a stack of eight Maven modules whose dependencies point only downwards, running as a single process with a small number of clearly assigned threads. No frameworks, servers or databases are involved. The text diagram in Appendix A shows the whole picture.

Layers and modules

Module	Contains	May depend on
`chronos-model`	Event, ids, event kinds, clocks (C4), canonical encoding and digests, world-image data types, fault-plan and intervention data types, and the LogReader and EventSink interfaces	nothing

Module	Contains	May depend on
`chronos-sdk`	The Node API (C1): node, node context, state codec, scenario, invariant and world-view accessor	model
`chronos-store`	Event store, segmented log, indexes, checkpoint files, timeline catalogue persistence and segment resolution, asynchronous writer (C6); implements EventSink and LogReader	model
`chronos-runtime`	Engine (C2), decision-source interface with the record-mode source only (C3), recorder (C5), fault injector (C12), invariant checker (C15), checkpoint capture and restore	sdk, model
`chronos-demo`	Demo applications and scenarios (C17)	sdk, model
`chronos-analysis`	Causal graph (C7), state reconstruction (C8), replay and branch decision sources, replay engine (C9), time-travel session (C10), timeline manager with id policy and branch creation (C11), counterfactual engine (C13), comparator (C14)	runtime, sdk, store, model
`chronos-cli`	Command-line interface, exporters, static HTML report (C16)	analysis, runtime, store, demo, sdk, model
`chronos-bench`	Benchmarks and stress harnesses (C18)	all

Tests live inside each module. Because Maven forbids cyclic module dependencies, the **single direction of dependency** (NFR-13) is enforced by the build itself. The demo module depends only on chronos-sdk and chronos-model, so a demo application can never reach into engine internals. The graph has no cycles, and a valid build order is model, sdk, store, runtime, demo, analysis, cli, bench.

Modules are created as phases need them (Section 24): the first week creates only `model`, `sdk`, `runtime` and `demo`.

Threads

Thread	Job	Rule
Engine thread (one per world)	Runs the step cycle and all node handlers	The only thread that touches a world; never blocks on disk
Writer thread (one per recording)	Takes event groups from a bounded queue and appends them to the log	Single writer, so no locks on the file; the queue gives back-pressure
Analysis worker pool	Runs counterfactuals and campaigns, one isolated world per job	Fixed size (default: number of cores); jobs share nothing mutable
Live-viewer server threads (Stretch; none in v1)	Serve JSON and pages to the browser	Read-only access to completed timelines

Only immutable data crosses thread boundaries (events, checkpoints, completed timelines). This design exercises threads, executors, queues and memory-visibility rules for real in a few well-defined places, which is a learning goal, while the engine itself stays single-threaded for determinism (D-02, D-21).

On-disk layout

```
chronos-data/
  catalog.json                     the timeline tree: ids, parents, status
  timelines/
    T0/
      header.json                  scenario, seed, nodes, fault plan, format
version, code fingerprint
      events/
        000000.log                 segment: one JSON event per line, step
groups closed by a commit line
        000001.log
      index/                       derived and rebuildable: by position,
node, kind, virtual time
      checkpoints/
        0001000.ckpt  0002000.ckpt ...
      summary.json                 final status, counts, end-state digest,
first violation
    T0.1/                          a branch: stores only events after its
branch point
      header.json                  includes parentId and branchSeq
      ...
```

Two storage rules: (1) the log is **append-only**; on startup, any trailing lines after the last commit line belong to an unfinished step and are discarded (NFR-11). (2) **Indexes are derived**: if they are missing or suspect, they are rebuilt from the log, so the log is the only thing that must be right.

Technology choices and why

Technology	Used for	Justification and alternatives rejected
Java 21	Everything	Records and sealed types model events and kinds cleanly; pattern matching simplifies analysis code. Virtual threads are *not* used in the engine, since scheduling must be ours
Maven	Build, modules, tests	Familiar and predictable; Gradle would also work. A tooling choice, not an architectural one

Technology	Used for	Justification and alternatives rejected
JUnit 5	Unit and integration tests	Standard
Jackson (core and databind)	JSON reading and writing	The only runtime library. Serialisation is infrastructure, not the intellectual core; we still own the canonical-form rules and the digests (D-09). Alternatives: Java built-in serialisation (brittle, unsafe format, not human-readable) and a hand-written JSON codec (educational but a time sink with real bug risk)
JMH	Micro-benchmarks	JVM benchmarking has well-known traps (warm-up, dead-code elimination); the standard harness avoids them. Build-time only
Inline SVG and plain JavaScript	Static HTML report	No framework and no server: the CLI writes one self-contained file that opens offline. A live viewer on the JDK's built-in HTTP server is Stretch (D-18)
Graphviz (optional, external)	Rendering DOT exports	Not a dependency; only a way to look at a file

Deliberately **not** used: Spring Boot, Kafka, Redis, any database, Kubernetes, Docker as an architectural element, Lombok, logging frameworks (CHRONOS uses the JDK's logger sparingly), command-line frameworks (the CLI is hand-written at first), mocking frameworks (hand-written fakes are enough). Each could be revisited only by a new entry in the decision log.

Scenarios

A scenario is a Java class registered under a name. It declares the node set and their configuration, the workload (scripted client inputs), the default fault plan, and the invariants. The run records the scenario name, parameters, seed and fault plan in the timeline header (D-22). Reason: the applications are Java anyway, and a second configuration language would add a layer with no benefit.

Cross-cutting rules

- **No banned APIs** in `model`, `runtime`, `demo` and sdk: wall-clock time, unseeded randomness, raw threads (the writer and worker pool live in `store` and `analysis`). A build-time scan enforces this (NFR-20).
- **Format versions** on every stored file (NFR-18).
- **Errors:** specific exception types (determinism violation, corrupt log, unknown timeline) with messages that cite event ids.
- **Identity everywhere:** components exchange ids (event, timeline, node), not object references into each other's memory.
- **Teaching notes:** every public interface has a short comment saying what it means in CHRONOS terms, because the team must be able to explain it.

24. Development phases

CHRONOS keeps the 17 phase numbers of the project plan as identifiers but builds them in a different order: first a thin spine that reaches the core idea (record, explain, branch, intervene, compare), then the deeper follow-ups. A suffix **a** marks the thin core of a phase and **b** its deeper follow-up. The table below is in execution order. Each step ends with working code, tests and a runnable demonstration before the next begins. Sizes are in joint weeks (S about 1, M about 2, L about 3 to 4) at roughly 10 to 15 focused hours per person per week; they are planning estimates to be recalibrated after Phase 4, when we know our real speed.

Step	Phase	What is built or learned	Exit criteria (demo and tests)	Weeks (cumulative)
1	1 Understand the problem	Distributed-systems basics, clocks, causality, replay; read this specification	Each of us explains happens-before, Lamport versus vector clocks, and determinism at a whiteboard; Appendix B decisions confirmed or amended	1 (1)

Step	Phase	What is built or learned	Exit criteria (demo and tests)	Weeks (cumulative)
2	2 Define the execution model	Interface sketches on paper (Node API, event fields, decision source, scenario); hand-simulated ping-pong and RE traces	Hand-made traces agree with what the code must later produce; design review passed	1 (2)
3	3 Smallest controlled environment	Maven skeleton (model, sdk, runtime, demo); Node API, engine, agenda, virtual clock, simulated network with fixed latency, timers, seeded generator; ping-pong runs	Ping-pong prints a trace; two runs print identical output; scheduler tie-breaking tests	2 (4)
4	4 Event model and recorder	Event types, deterministic ids, in-memory recorder, basic invariants without witnesses, CLI `run` and determinism check	Identical log hashes over 100 seeds; golden-file test for ping-pong	2 (6)
5	5 Logical clocks and causal tracking	Lamport and vector clocks with N + 1 slots, parent edges with kinds, `env` follows the node rules, clock values carried on every edge kind	Section 11 examples pass as tests; property tests on random runs	2 (8)
6	6 Causal graph	Graph builder, queries, vector-versus-reachability self-check, ASCII space-time diagram, DOT export	`why` on an RE-lite run prints the causal path; self-check passes on 10,000 random runs	1 (9)
7	7 Event persistence	Segmented JSON-lines log, commit lines, header, catalogue, indexes, asynchronous writer, LogReader and EventSink implementations	Kill-the-process test leaves a readable log; reload equals original; throughput measured	2 (11)

Step	Phase	What is built or learned	Exit criteria (demo and tests)	Weeks (cumulative)
8	8 State snapshots and reconstruction	State contract with volatile and durable zones (zone tag reserved), state codec, canonical encoding, whole-state and per-field digests, state records, last-writer map, recording accessor, WITNESS edges, checkpoints, restore	Reconstruct equals live at every step; snapshot round trip equals uninterrupted run; violation events carry witness edges	2 (13)
9	9 Deterministic replay	Replay decision source, replay engine, verifier, mismatch report, prefix replay, the deliberately bad node	AD-2 complete; replay from every checkpoint equals replay from the start; prefix replay rebuilds a world at any step	3.5 (16.5)
10	10a Minimal time travel	Session, cursor, `goto`, `step`, `back`, `state`, `world`, `inflight`, `timers`, `why`	AD-3 runs live; reversibility and fidelity tests pass	1 (17.5)
11	11 One branch	Timeline manager (id policy, branch creation), one branch from a step boundary with prefix replay, shared prefix, catalogue	Prefix identity test; a no-op branch is event-identical to its parent	2 (19.5)
12	12a Partition fault	Fault plan with the at-time trigger; partition and heal; fault events and edges; OMISSION edges for partitions; coverage report	RE with a 400 ms partition reproduces the split brain (AD-1); plan plus seed gives identical logs	1 (20.5)

Step	Phase	What is built or learned	Exit criteria (demo and tests)	Weeks (cumulative)
13	13a One intervention, one verdict	One intervention type (remove the partition or change its heal time), INTERVENTION_APPLIED, a single run, verdicts FAILURE_DISAPPEARS, FAILURE_PERSISTS, NO_EFFECT and INCONCLUSIVE; no worker pool	AD-4; a no-op gives NO_EFFECT; locality property test for plans without nonLocal rules	1.5 (22)
14	14a Comparator core	Alignment, classes, per-timeline taint and roots, first causal divergence, explanation path with the anchor rule, divergence report	AD-5; KA-1 and planted-perturbation tests; anchor rule validated with a hand-built cross-timeline case; laws M7, M9, M10	2 (24)
15	10b Deeper time travel	`next-on`, `prev-on`, `find`, `effects`, `concurrent-with`, `watch`, `diff`, `run-to`, breakpoints and stepped replay	Each command tested; a breakpoint stops at exactly the right event; `diff` agrees with the comparator	1 (25)
16	12b Full fault catalogue	Drop, delay, crash, restart with the durable zone, duplicate, reorder, pause, clock skew, slow link; the other triggers; OMISSION scopes for crash and pause; sequential seed campaigns	A campaign over seeds finds a failing RE run; KA-2 and KA-3 reproduce; the durable zone survives a restart	1 (26)

Step	Phase	What is built or learned	Exit criteria (demo and tests)	Weeks (cumulative)
17	13b Full counterfactuals	The rest of the intervention catalogue, DIFFERENT_FAILURE, the parallel worker pool, campaigns moved onto the pool	AD-4 for several interventions; reports identical across thread counts (M12); KA-2 and KA-3 confirmed by counterfactual	2 (28)
18	14b Comparator hardening	Absorbed divergence, suspected-nondeterminism flag, concurrent roots, KA-4	Hand-built two-root case; KA-4 reports FAILURE_PERSISTS and lists both causes	1.5 (29.5)
19	15a Static HTML report	DOT and JSON exports, then the self-contained HTML report: inline SVG space-time diagram, event table, causal path, divergence highlighting, optional hover and filter	AD-6; the report opens offline and makes no network calls	2 (31.5)
20	16 Stress test and benchmark	JMH benchmarks, 1,000,000-event runs, 10,000-seed chaos soak, memory profile, fix the worst bottlenecks	Report comparing measurements with the targets in Section 8	2 (33.5)
21	17 Documentation, demonstrations, evaluation	README, user guide, architecture document, demo scripts, evaluation chapter, viva preparation, final decision log	All six acceptance demonstrations run from a clean checkout in under 10 minutes	2 (35.5)

Milestones

Milestone	Steps (phases)	What it means
A. The recorder	1 to 6 (Phases 1 to 6)	A controlled run is recorded with clocks and causality, and you can ask *why*
B. The memory	7 to 8 (Phases 7 and 8)	Runs persist; any past state can be reconstructed; violations carry witnesses
C. Verified replay	9 (Phase 9)	The recording is provably reproducible
S. The core idea	10 to 14 (Phases 10a, 11, 12a, 13a, 14a)	Record, explain, branch, intervene, compare: AD-1, AD-3, AD-4 and AD-5 run on RE with minimal time travel, one partition fault, one intervention type, one branch and one comparison. About 24 joint weeks
D. Depth	15 to 18 (Phases 10b, 12b, 13b, 14b)	Full time-travel commands, the full fault catalogue, more interventions, parallel counterfactuals, hardened comparison
E. Show and prove	19 to 21 (Phases 15a, 16, 17)	Static HTML report, measurements, documentation and evaluation

Fault-injection *hooks* (the point in the step cycle where a fault may alter an item) are built into the engine in Phase 3, even though the faults themselves arrive in Phases 12a and 12b. This avoids reopening the engine later.

Demonstrations by step. AD-2 arrives in Phase 9 and AD-3 in Phase 10a. AD-1 arrives in Phase 12a, because the failing RE run needs the partition (the invariant check starts in Phase 4 and witness edges in Phase 8). AD-4 arrives in Phase 13a, AD-5 in Phase 14a and AD-6 in Phase 15a. Milestone S is the proof of concept of the whole project, so if the schedule slips, Milestones D and E shrink before any step of Milestone S does.

Definition of done for every phase

1. The code builds and all tests pass.
2. At least one property, determinism or known-answer test exists for the new mechanism.

3. A runnable demonstration exists and is written down as a short script.
4. The decision log is updated and any deviation from this specification is recorded.
5. A short teaching note exists, and the author has explained the subsystem to the other team member.

Cut lines

If time runs short, cut in this order and keep everything above the line: (1) intervention search; (2) the live viewer; (3) pause, clock-skew and reorder faults; (4) the knowledge view and `watch`; (5) node-local replay; (6) log compression. **Never cut:** the replay verifier, branching, the comparator, and the known-answer tests; they are the proof that the core idea works.

Suggested working split

Phases 1 to 5 are done together, because everyone must understand the foundation. From Phase 6, one possible split is **Track A, the record side** (persistence, state, replay, branching, faults) and **Track B, the analysis side** (causal graph, time-travel commands, comparator, HTML report). The tracks meet at Phases 9, 11 and 13a. Swap tracks after Milestone S (Phase 14a) so each of us learns both halves. This is a suggestion for the two of you to confirm, not an assignment. Each subsystem is reviewed by the other person before it is considered done: one Git repository, feature branches, and cross-review before merging (Q8).

Estimate

The sizes add up to about **35.5 joint weeks** (v0.1 said 37; the difference is Phase 15, which shrinks from 3.5 to 2 weeks because the web viewer became a static HTML report). **Milestone S is reached at about 24 joint weeks.** Working in parallel from Phase 6 may reduce the total to roughly 27 to 31 calendar weeks. These numbers depend on your real weekly hours and your deadline, which this specification does not yet know: Q1 is still open (Appendix B).

25. Testing strategy

CHRONOS's central claims are exact (replay is identical, clocks are correct, the named divergence really is first), so most of our tests check *laws* and *known answers* with exact oracles rather than loose smoke checks. A bug in a debugger is especially costly, because it sends the developer hunting in the wrong place.

Test layers

Layer	What it checks	Examples
Unit	Small pure mechanisms	Lamport and vector rules, vector comparison, our random generator, canonical encoding, agenda ordering and tie-breaking
Property (seeded, randomised)	Laws over many random runs	Generate random message-passing scenarios from a seed and check the laws below. A failing seed is printed and kept as a regression case
Determinism	Repeatability	Run each scenario twice in one JVM and in two separate JVM launches; compare log hashes byte for byte
Golden file	Accidental change	Checked-in logs for ping-pong and RE-lite; any difference must be a conscious, reviewed decision
Integration	Pipelines	Record, persist, load, reconstruct, replay, branch, compare
Known answer	Analysers tell the truth	Scenarios with a planted root cause (below)
Chaos soak	Robustness	Thousands of seeds with random fault plans, checking the laws on every run
Concurrency stress	Writer and worker pool	Many iterations of the recorder and of parallel counterfactuals; results independent of thread count
Crash safety	Durable log	Kill the process mid-write; truncate or corrupt files; expect recovery or a clear refusal
Negative	CHRONOS does not lie	The bad-citizen node must be flagged; a corrupt or wrong-version log must be refused
End to end	The demonstrations	AD-1 to AD-6 as automated scripts

We write our own small seeded generator for property tests instead of adding a library. It reuses the same random generator as the runtime, so every failing case is reproducible from its seed.

The laws CHRONOS must obey

Law	Statement	Related requirement
M1 Replay	Replaying any recorded timeline reproduces its events exactly	NFR-01, FR-22, FR-23
M2 Reconstruction	State rebuilt from the log equals the state seen live at every step	NFR-03, FR-20
M3 Clocks	X happened before Y exactly when vector(X) is less than vector(Y); and if X happened before Y then Lamport(X) is less than Lamport(Y), counting FAULT, WITNESS and OMISSION edges as well as messages	NFR-02, FR-11, FR-12
M4 Graph	Every edge points to an earlier position; reachability equals vector order	FR-13 to FR-15
M5 Snapshot	Restoring a checkpoint and running on equals the uninterrupted run	FR-21
M6 Prefix	A branch's events up to its branch point equal its parent's	FR-29
M7 No-op	A branch with no interventions equals its parent, and, once the comparator exists, comparison reports no divergence	FR-29, FR-41
M8 Locality	Nodes with no causal connection to an intervention behave identically in the branch, for fault plans with no nonLocal rules (D-13)	Section 19
M9 Identity	Comparing a timeline with itself reports no divergence	FR-39

Law	Statement	Related requirement
M10 Root	For a timeline and its branch made by intervention I, the first causal divergence is I's own INTERVENTION_APPLIED event (D-25), and every root lies in I's causal future	FR-41
M11 Immutability	Files of a completed timeline never change	NFR-04
M12 Thread independence	Counterfactual and campaign reports do not depend on the number of worker threads	FR-36
M13 Environment identity	Adding or removing one event on env in a branch does not change the id of any other env event	FR-07 (D-24)
M14 Witnesses	The witnesses of an INVARIANT_VIOLATED event are exactly the last writers of the state fields its invariant read	FR-44 (D-17)

Each law must have at least one automated test before the phase that introduces it is declared done.

Known-answer bug library

The analysers can only be trusted if they find causes that we planted and therefore know.

Scenario	Planted cause	What CHRONOS must report
KA-0 Control	None: a correct implementation under thousands of random fault plans	No violation, ever; proves CHRONOS does not hallucinate bugs
KA-1 RE split brain	A 400 ms partition longer than the 300 ms watchdog	Root is the partition event; the chain runs through the promotion to the violation; healing early makes the failure disappear
KA-2 Lost update	A replication message delayed past its successor, with no sequence numbers	Root is the delay; the but-for test confirms it

Scenario	Planted cause	What CHRONOS must report
KA-3 Blocked commit	Coordinator crashes after prepare in a two-phase commit	Root is the crash; with no crash, no participant stays undecided
KA-4 Overdetermined	Two independent partitions, each enough to break the system	Removing either alone reports FAILURE_PERSISTS, as documented in Section 19

Blind debugging protocol

Planted bugs can bias our tests. To reduce that, one of us writes or seeds a bug in a scenario *without telling the other*, and the other must find the cause using only CHRONOS. We record whether the tool led to the right diagnosis, how long it took, and what was confusing. These sessions become part of the evaluation chapter (Phase 17). We also run campaigns on scenarios to find failures we did not plan, and diagnose them by hand and with the tool to compare answers.

Analyser mutation tests

- **Log perturbation:** copy a log, change one event, and require the verifier to fail at exactly that event and the comparator to name exactly that event as the root.
- **Injected nondeterminism:** add a hidden use of time or hash order to a node and require the determinism checks to catch it.

Practical rules

- Every randomised test prints its seed; failing seeds are added to a regression list in the repository.
- Tests run on every commit locally; if the repository host offers automatic builds, we use them, with no extra technology.
- Line coverage is reported but not worshipped. The real gate is that every law above has a test.
- The author writes the tests; the other team member reviews them as carefully as the code.
- Visual output (the HTML report) is checked by a short manual checklist per release, since pixel correctness is not part of the core claims.

26. Benchmarking strategy

Benchmarks answer specific questions with numbers: does CHRONOS meet the targets of Section 8, and where does it bend or break? They start in the phase that builds each subsystem, so surprises arrive early, and Phase 16 consolidates them into one report.

Principles

1. **Question first.** Each benchmark is written as a question and a *prediction* before anything is measured. Writing predictions down trains our intuition for where the time goes.
2. **Fixed seeds and fixed workloads,** so a number can be reproduced and compared.
3. **Two tools.** JMH for small mechanisms (clock operations, encoding), where JVM warm-up and optimisation make naive timing misleading. A plain timing harness with repeated runs for whole-system measurements.
4. **Honest statistics.** At least five repetitions, the first discarded for warm-up; report median, 95th and 99th percentile, not just the mean.
5. **Record the environment:** processor, memory, operating system, JDK version, heap size.
6. **Results are kept as CSV in the repository,** with charts generated for the Phase 16 report. Misses against targets are reported, not hidden.

Workload sizes

Size	Events per timeline	Node counts
S	1,000	3
M	10,000	3, 5
L	100,000	5, 10
XL	1,000,000	10, 20

Beyond RE, we use a synthetic *gossip* scenario in which nodes exchange randomly chosen messages, so we can scale events and nodes freely.

Benchmark catalogue

ID	Question	What varies	Metric	Target
B1	How fast does the bare engine step, with recording off?	Node count, fan-out	Steps per second	Baseline only
B2	What does recording cost?	Recorder off, in-memory, synchronous disk, asynchronous disk	Events per second; overhead factor	NFR-05
B3	How does vector-clock cost grow with the node count?	3, 10, 20, 50, 100 nodes	Nanoseconds per operation; bytes per event	Informs compression
B4	How big and how slow is event encoding?	With and without state records	Bytes per event; microseconds per event	NFR-10
B5	How do the causal graph build and queries scale?	Timeline size; size of the causal past	Build time; query latency; memory	Section 13
B6	How fast can we seek, and what checkpoint interval is best?	Timeline size; interval 100, 1,000, 10,000	Seek latency	NFR-06
B7	How fast is verification replay?	Timeline size	Events per second; ratio to original run	NFR-07
B8	What does branching cost, and does sharing the prefix pay off?	Prefix length; shared versus full copy	Branch creation time; disk used	NFR-08
B9	How does counterfactual throughput scale with threads?	1, 2, 4, 8, all cores	Runs per second; speedup; memory per job	FR-36

ID	Question	What varies	Metric	Target
B10	How does comparison scale?	Event count	Time; memory	Section 20
B11	How long does intervention search take? (If built)	Number of suspects	Total time	Stretch
B12	How much disk does a timeline use?	With and without full state records	Megabytes per million events	NFR-10
B13	How does the asynchronous writer behave under a slow disk?	Queue size; simulated delay	Back-pressure behaviour; queue length	NFR-05
B14	How long do the acceptance demonstrations take?	AD-1 to AD-6	Wall-clock time	Under 10 minutes in total (Phase 17 criterion)

When each benchmark first runs

Phase 3: B1. Phase 4: B2 (in-memory). Phase 5: B3. Phase 6: B5. Phase 7: B2, B4, B12, B13. Phase 8: B6. Phase 9: B7. Phase 11: B8. Phase 13b: B9 (and B11 if built). Phase 14a: B10. Phase 16: everything at XL size. Phase 17: final numbers for the report and the viva.

Results change decisions

Benchmarks are only worth running if their outcomes trigger actions. These are the rules we commit to in advance:

If we observe	Then we
Persisted recording below 20,000 events per second	Profile; look first at encoding, then at the writer queue
Seek takes over 1 second at 100,000 events	Shorten the checkpoint interval or improve the indexes
Vector-clock cost dominates at 20 nodes	Implement delta compression of vectors

If we observe	Then we
Storage exceeds about 1 KB per event	Revisit the state record policy (D-10): deltas or periodic snapshots
Counterfactual speed-up stops well below the core count	Look for shared mutable state or memory pressure
Replay is more than twice as slow as recording	Profile the verifier's comparison and hashing

27. Risks and difficult areas

The three hardest things in CHRONOS are keeping replay truly deterministic, getting counterfactual and first-divergence semantics exactly right, and making checkpoints complete enough that a restored world behaves identically. Most other risks are about scope and learning, not about technique.

The three hardest areas

1. **Deterministic hygiene.** One stray use of time, one hash-ordered collection of enums, one static variable, and replay breaks in a way that appears only on another machine. *Defence:* the determinism contract, the banned-API scan, determinism checks from Phase 4, tests across two JVM launches, and the deliberately bad node.
2. **Counterfactual and divergence semantics.** It is easy to build something that prints plausible answers. It is harder to build something provably correct. *Defence:* the formal definitions in Sections 19 and 20, the locality and root laws (M8, M10), known-answer scenarios, and blind debugging sessions.
3. **Checkpoint completeness.** If a checkpoint forgets one piece of world state (a fault rule's progress counter, a per-sender message ordinal, the last-writer map, a random stream position, a pending timer), a branch looks fine for a while and then diverges for no visible reason. *Defence:* a written checklist of every world-state item (Section 14), and the snapshot law M5 tested at every checkpoint of every demo run.

Risk register

#	Risk	Why it is dangerous	Likelihood and impact	Mitigation	Early warning
R1	Scope explosion	The idea invites endless extras and polish	High, high	Non-goals (Section 5), phase gates, cut lines (Section 24)	A phase takes more than 1.5 times its estimate
R2	Hidden nondeterminism	Breaks replay silently	Medium, high	Contract, scan, early determinism checks, bad-citizen test	Log hashes differ between two JVM launches
R3	Noisy counterfactuals	Unrelated differences hide the real effect	Medium, high	Identity-keyed randomness, stable tie-breaking, locality law	Unaffected nodes differ in a branch
R4	State serialisation fidelity	Wrong digests, false divergences	Medium, medium	Simple state types, canonical-form tests, round-trip tests	Digest mismatch after restore
R5	Wrong or unconvincing first-divergence results	The headline feature would mislead	Medium, high	Formal definition, planted perturbation tests, concurrent-root case, blind protocol	Roots outside the intervention's causal future
R6	Event volume and storage growth	Slow runs, huge logs	Medium, medium	Scale envelope, state record policy, early benchmarks B4 and B12	More than 1 KB per event
R7	Vector clocks at many nodes	Cost grows with node count	Low, low	Limit of 20 nodes; delta compression if needed	B3 shows dominant cost
R8	Steep learning curve	We might build what we cannot explain	Medium, high	Teaching notes, Phase 1 gate, explain-to-partner rule, reading list	Cannot explain a component at review
R9	Demo applications too trivial or too complicated	Boring or unmanageable showcase	Medium, medium	Tiered demos; keep RE small enough to hold in your head	Demo code needs a new feature in the runtime to work

#	Risk	Why it is dangerous	Likelihood and impact	Mitigation	Early warning
R10	Visualisation time sink	Frontend work swallows the schedule	High, medium	Text and DOT first, then a static HTML report with no framework (Phase 15a); the live viewer is Stretch	Report or viewer work starts before Milestone S is done
R11	Stored formats change	Old logs unreadable	Medium, low	Format version on every file; runs are reproducible from their seed, so logs can be regenerated	Golden files change unexpectedly
R12	Concurrency bugs in writer or worker pool	Flaky, hard-to-diagnose failures	Medium, medium	Single writer, immutable data, stress tests, engine stays single-threaded	**Any flaky test** (in a determinism project, flakiness is an alarm)
R13	Over-engineering	Interfaces and layers with one implementation and no purpose	Medium, medium	Vertical slices; an interface only where two implementations exist or a test needs a fake; no placeholder presented as complete	Abstractions nobody calls
R14	Two people drifting apart	Different terms, duplicated or conflicting work	Medium, medium	This specification as single source of truth, decision log, project journal in the repository, branch per feature, cross-review	Same concept named two ways
R15	Evaluation bias	Planted bugs are found because we planted them	Medium, medium	Blind protocol, unplanned discovery runs, control scenario KA-0	Perfect results on every scenario
R16	Platform differences	Line endings, locale, file separators break determinism	Low, medium	UTF-8 everywhere, fixed newline, fixed locale, tests on every operating system we use	Golden files differ across our machines

#	Risk	Why it is dangerous	Likelihood and impact	Mitigation	Early warning
R17	Unknown deadline	Wrong plan for the real time budget	Medium, high	Confirm the deadline now; Milestone S (about 24 joint weeks) is built first, so the core idea exists before any depth is added	Deadline conversation not yet held
R18	Explanation-path rules not yet validated	The OMISSION scope and the cross-timeline anchor rule are new; a wrong path misleads the developer	Medium, high	Define them in Phases 12a and 14a; validate with KA-1 and a hand-built cross-timeline case; fall back to certain edges only if they fail	The path skips the promotion, or starts in the wrong place

28. What should be implemented first

Build the smallest closed loop first: the ping-pong run on a deterministic engine, recorded as events with Lamport and vector clocks, printed as a space-time diagram, and proven repeatable. Persistence, replay, branching and any user interface come after that loop works and is tested.

Week 0: before any code

1. Read this specification together in one sitting; each of us marks every unclear paragraph, and we resolve them out loud.
2. Confirm or amend decisions D-01 to D-26 and answer the open questions in Appendix B.
3. Hand-simulate the ping-pong trace and the three-node worked example from Section 11 on paper, computing every clock value ourselves (Phase 2).
4. Agree repository rules: one Git repository, a branch per feature, cross-review before merging, and a short project journal file recording what each of us did and decided.
5. Confirm the deadline and our real weekly hours, so Section 24's estimate can be recalibrated.

6. Install JDK 21 and Maven on both machines and check that an empty multi-module project builds on both.

Slice 0: ping-pong, recorded and explained

This slice is Phases 3 to 6 in miniature. It is done when all of the following are true:

- Two nodes, A and B, run on the engine with 10 ms latency and a fixed seed, with no threads.
- The engine uses an agenda, virtual time and stable tie-breaking.
- Every effect is an event with identity, position, virtual time, Lamport value, vector clock and parents.
- The program prints the event table of Section 10 exactly, including the env vector slot, and a golden-file test locks it.
- The program prints the space-time diagram of Section 13.
- Running it twice produces byte-identical output.
- Tests pass for the worked example of Section 11, for vector order versus graph reachability on random chains, and for tie-breaking stability.

Components involved: C1, a minimal C2, C3 (record mode with fixed latency), C4, C5 (in-memory only), a minimal C7, and plain printing from C16. **Not yet:** store, state records, checkpoints, faults, replay.

Build order inside Slice 0

1. Event, event id and event kind data types, with a plain canonical text form (model).
2. Lamport and vector clocks, tested against the Section 11 examples (model).
3. The seeded random generator and the identity hash used for identity-keyed decisions (model).
4. The agenda with stable ordering and the virtual clock, with tests (runtime).
5. The Node API, the node context, and the engine's step cycle for start, send and deliver (the Node API and context in sdk, the step cycle in runtime).
6. The in-memory recorder and event stamping (runtime).
7. Ping-pong in the demo module and its golden-trace test.
8. The space-time diagram printer and a first `why` query.
9. The determinism-check command.

The next slices

- **Slice 1:** timers, RE-lite (primary and backup with heartbeats, no faults yet) and a basic invariant. This brings in everything needed for Phase 4 and 5 completion.
- **Slice 2:** persistence and state records (Phases 7 and 8).
- **Slice 3:** the replay verifier and the deliberately bad node (Phase 9), which completes Milestone C.

Slice 4 (Milestone S): the thin spine, Phases 10a, 11, 12a, 13a and 14a: minimal time travel, one branch, the partition fault with OMISSION edges, one intervention with a verdict, and the comparator core. After it, AD-1, AD-3, AD-4 and AD-5 run on RE.

What not to do first

Persistence, any web interface, counterfactual logic, performance tuning, generic plug-in interfaces, clever serialisation, and anything involving threads. Each would be built on a foundation we have not yet proved.

Questions to settle before Phase 3

Settled in v0.2: Maven (Q2); Jackson as the only runtime library (Q3); RE as the flagship demo (Q4); real-network execution out of v1 (Q5, D-03); a declared version string plus a best-effort class hash (Q7); one Git repository with feature branches and cross-review (Q8); KA-2 and KA-3 as the extra known-answer scenarios (Q9); node states under about 10 KB (Q10). **Still open:** the deadline and weekly hours (Q1) and the form of evaluation your institution expects (Q6). Both should be answered before Phase 3, because they can change the plan.

Pre-reading

This short list is cited from memory; check titles, years and venues before using them in the project report.

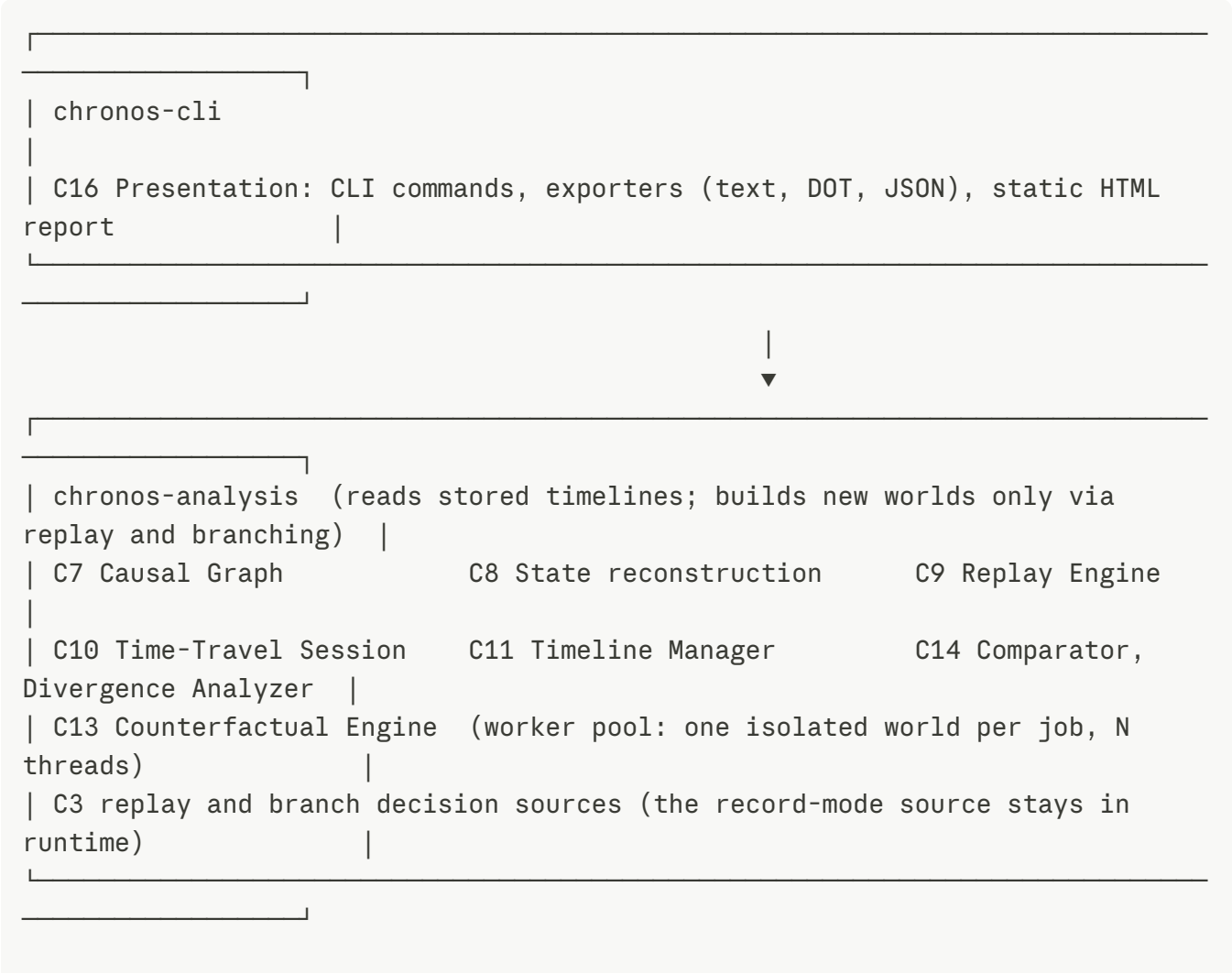
- Leslie Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System* (1978): the origin of happens-before and Lamport clocks.
- Colin Fidge (1988) and Friedemann Mattern (1989) on vector time: the origin of vector clocks.
- K. Mani Chandy and Leslie Lamport, *Distributed Snapshots* (1985): consistent global states and why CHRONOS gets them for free.

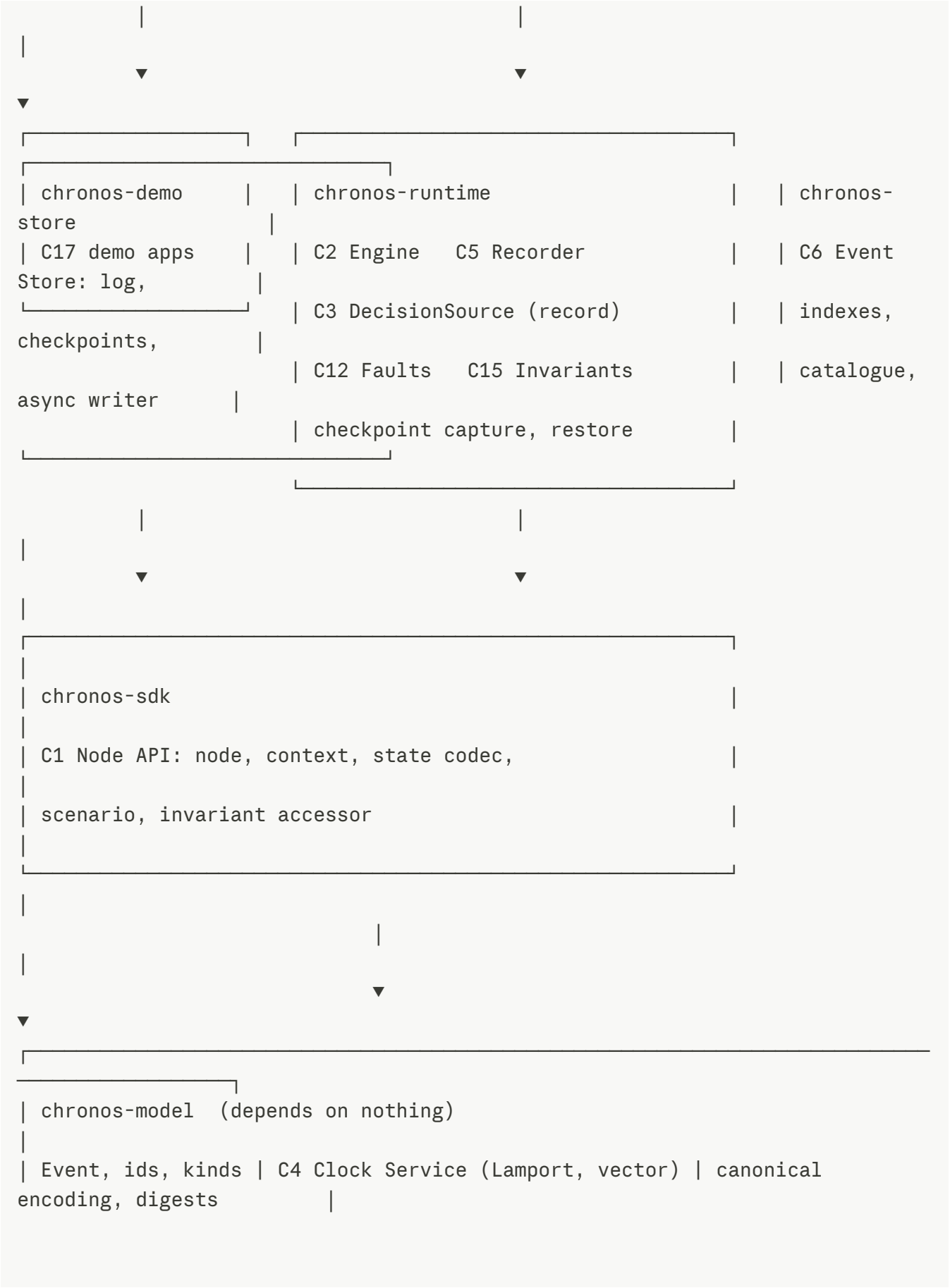
- Martin Kleppmann, *Designing Data-Intensive Applications*: the chapters on the troubles of distributed systems and on consistency.
- Ivan Beschastnikh and colleagues, *Debugging Distributed Systems* (ACM Queue, 2016): the case for visualising causal order.
- Will Wilson's talk on testing distributed systems with deterministic simulation, as practised at FoundationDB (Strange Loop, 2014): the closest relative of our controlled execution model.

Appendix A: High-level architecture diagram

The first diagram shows the module stack and the single direction of dependency (arrows point to what a layer depends on). The second shows what happens inside one recording run.

A.1 Module stack



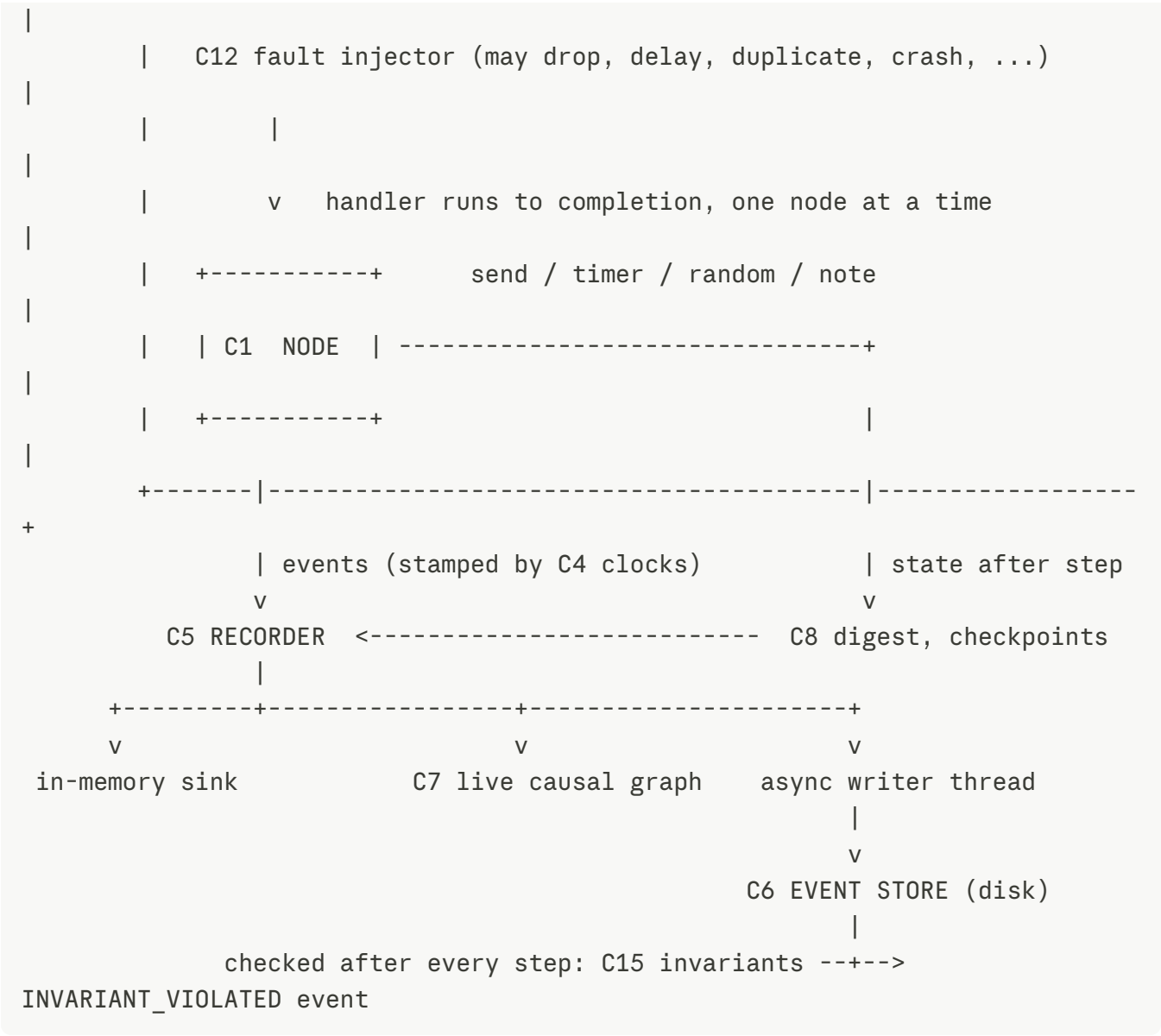

chronos-demo
C17 demo apps
chronos-runtime
C2 Engine
C5 Recorder
C3 DecisionSource (record)
C12 Faults
C15 Invariants
checkpoint capture, restore
chronos-store
C6 Event Store: log, indexes, checkpoints, catalogue, async writer
chronos-sdk
C1 Node API: node, context, state codec,
scenario, invariant accessor
chronos-model (depends on nothing)
Event, ids, kinds
C4 Clock Service (Lamport, vector)
canonical encoding, digests

```
| world-image types | fault-plan and intervention types | LogReader and
EventSink interfaces   |
└──────────────────────────────────────────────────────────────────────────────
──────────────────┘
```

The picture shows the main layering only; the table in Section 23 lists every allowed dependency. `chronos-demo` depends only on `chronos-sdk` and `chronos-model`, so a demo application can never reach into engine internals. `chronos-runtime` depends on `chronos-sdk` and `chronos-model`. `chronos-analysis` depends on runtime, sdk, store and model. `chronos-cli` depends on analysis, runtime, store, demo, sdk and model. `chronos-bench` depends on every module, and tests live inside each module (omitted for clarity). The analysis layer never calls into a running engine; it creates new worlds only through replay and branching.

A.2 Inside one recording run

```
                        scenario + seed + fault plan + horizon
                                      |
                                      v
        +-------------------------------------------------------------------
+
        |                       C2  SIMULATION ENGINE
|
        |
|
        |   agenda (deliveries, timers, inputs, fault actions)
|
        |        |
|
        |        v        asks
|
        |   scheduler  <----------->  C3 DECISION SOURCE
|
        |        |                     record: seeded policy + fault plan
|
        |        |                     replay: answers read from the log
|
        |        |                     branch: parent log, then policy +
|
        |        |                             interventions
|
        |        v
```



A.3 How the four flows use the stack

Flow	Starts at	Passes through	Ends at
Record	CLI and scenario	Engine, decision source (record mode), clocks, recorder	Store: an immutable timeline
Inspect	CLI command	Time-travel session, store indexes, state reconstruction, causal graph	Text or picture; nothing written

Flow	Starts at	Passes through	Ends at
Branch	CLI or counterfactual engine	Timeline manager, replay (prefix), engine with branch decision source	Store: a child timeline and a verdict
Compare	CLI or counterfactual engine	Store, causal graph, comparator	A divergence report

Appendix B: Architectural decision log and open questions

This log is the shared memory of the project. Every entry is a *recommendation* until both of us confirm it. After confirmation, a decision is changed only by adding a new decision that supersedes it, with its consequences written down.

Implementation status: specification v0.2 (this document); Phases 1 to 17 not started and no code written.

Decision log

ID	Decision	Alternatives considered	Why this one	Section	Status
D-01	Applications run on a controlled runtime through the Node API	Bytecode agents; JVM-level record and replay; operating-system-level recording	Feasible for two students; gives control of time, network, ordering and randomness	3, 9	Approved
D-02	One single-threaded deterministic discrete-event engine per world	Baton-passing threads; free threads with recorded interleaving	Simplest route to exact determinism and snapshots	9	Approved
D-03	Simulation mode only in v1; real-network execution is out of scope. A live-network recorder is a possible post-v1 idea and must never enter the replay core	Live processes over TCP (record only)	Replay would not be deterministic; large extra scope	9	Approved, re-worded in v0.2

ID	Decision	Alternatives considered	Why this one	Section	Status
D-04	Node memory lives in one serialisable state object with a volatile zone (lost on crash) and a durable zone (survives a crash, stands for disk). Each node declares a state codec through the Node API (state type, canonical encode and decode). Durable changes commit at step end, atomically with the step's events. Phases 3 to 11 are volatile-only and crash-stop; the durable zone and restart arrive in Phase 12b	Heap snapshots via built-in serialisation; reflective field scanning; a separate simulated disk object	Makes snapshots, digests and comparison reliable, and makes restart semantics explicit	9, 10, 14, 18, 21	Amended in v0.2
D-05	Events are recorded at handler level: a trigger plus effect events per step	Method-level events; message-only events	Matches the runtime; captures every decision; small	10	Approved
D-06	Identity-keyed randomness, per-node streams, our own generator, stable tie-breaking	One shared random stream; the JDK generator; insertion-order tie-breaking	Prevents butterfly noise; gives the locality property	9, 19	Approved
D-07	Lamport and vector clocks on every event; fixed node set per run; vectors have N + 1 slots (nodes in declared order, `env` last) and every table, diagram, golden file and stored event shows all slots	Vector only; sparse dynamic vectors; hiding the `env` slot when it is zero	Cheap, teachable, exact; one format everywhere avoids mismatches between display and storage	10, 11	Amended in v0.2
D-08	Event id is node plus per-node counter; message id is its send event id; content digest excludes position. Extended by D-24 (env ids) and D-25 (origin in the digest)	Global counters; random identifiers	Stable identity across timelines is what makes comparison possible	10	Approved; extended by D-24, D-25

ID	Decision	Alternatives considered	Why this one	Section	Status
D-09	Canonical JSON written with Jackson, the only runtime library; we own canonical rules and digests	Built-in serialisation; hand-written codec; binary formats	Human-readable and diff-friendly; serialisation is infrastructure	23	Approved
D-10	Digest every step; full post-state on change; policy knob for later change. The byte-per-event figure of NFR-10 is measured, not a pass-or-fail target	Deltas; periodic snapshots plus re-execution	Simplest; reconstruction needs no application code	14	Approved
D-11	Append-only segmented JSON-lines log with commit lines, derived indexes, checkpoint files; no database. The store owns catalogue persistence and segment resolution	Embedded SQL database; custom binary format	The log is the event-sourcing idea itself; easy to inspect and recover	17, 23	Approved
D-12	Timelines are immutable; a branch references its parent's prefix	Full copies; persistent data structures	Cheap branching; explicit sharing	17	Approved
D-13	Faults are first-class, deterministic, recorded, applied only by the engine; declared as fault plans. Every rule is keyed by time or identity. Triggers: at a virtual time; after a named event; on a named message id; the nth message of one type from one named sender to one named receiver (counted in the sender's history and checkpointed with it); probabilistic by hash of (seed, message id, rule id); on node state (Stretch). No global counters; maximum firings only on time-triggered or event-triggered rules; a wildcard sender only if flagged nonLocal, which voids law M8 for that run	Faults inside node code; external chaos tools; global message counters	Repeatability, causal traceability and the locality property	14, 18, 19, 25	Amended in v0.2

ID	Decision	Alternatives considered	Why this one	Section	Status
D-14	Causal graph implemented by us on dense arrays	Graph library or graph database	Simple DAG; learning goal; speed	13	Approved
D-15	Counterfactual continuation uses the Continue policy; Pinned is deferred	Pinned replay of the original order	Continue already repeats unaffected timings	19	Approved
D-16	The first causal divergence is the earliest causal root. Align by event id; compute taint separately inside each timeline over its own causal graph (a root is a divergent event with no tainted parent); collect the roots of both timelines; order them by virtual time, then node order (`env` last), then event id; never compare Lamport or vector values across timelines; within one timeline a root that happened before another comes first; report all roots with the earliest as primary. The explanation path is computed in the timeline that contains the failure, from the primary root if it lives there, otherwise from the root's anchor: the last Same event on the root's node before the divergence. The anchor rule is validated in Phase 14a	First differing log position; textual diff of the global order; comparing clocks across timelines	Log position is arbitrary among concurrent events, and clocks of different executions are not comparable	20, 21	Amended in v0.2

ID	Decision	Alternatives considered	Why this one	Section	Status
D-17	Invariants are checked globally by the engine after every step; a violation is recorded as INVARIANT_VIOLATED on `env`. The engine records the last-writer event of every state field (per-field digest comparison after each step). Invariants read state through a recording accessor; the witnesses are the last writers of exactly the fields read. The violation event has the step trigger (STEP_EFFECT) and one WITNESS edge per witness as parents, and the `env` vector merges all witness vectors before it increments. Basic check in Phase 4, witnesses in Phase 8	Per-node checks only; after-the-fact log scans; a per-node instead of per-field last writer	The engine has a god's-eye view; field level gives precise, honest causes	10, 12, 14, 21	Amended in v0.2
D-18	CLI text first; then DOT and JSON exports; then a static, self-contained HTML report generated by the CLI (inline SVG space-time diagram, event table, causal path, divergence highlighting, optional hover and filter; no network calls). A live local viewer on the JDK HTTP server is Stretch (post-v1)	JavaFX or Swing; a web framework; a live viewer in v1	Needs no server or framework, opens offline, keeps the schedule short	4, 13, 21, 23, 24	Amended in v0.2
D-19	A decision source is the single seam for nondeterminism; one engine serves record, replay and branch; no separate decision log. The interface and the record mode live in runtime; the replay and branch modes live in analysis and read logs through LogReader	A separate replay engine	One code path means replay correctness is a property of the engine	9, 21, 23	Approved; placement clarified

ID	Decision	Alternatives considered	Why this one	Section	Status
D-20	Java 21, Maven, JUnit 5, JMH; hand-written fakes; our own seeded property-test generator	Gradle; mocking and property-testing libraries	Minimal dependencies; reproducible failing cases	23, 25	Approved
D-21	Threads: engine single-threaded per world, one asynchronous writer, a fixed worker pool for counterfactuals; viewer server threads only if the Stretch live viewer is built	Asynchronous everything	Concurrency only where it is safe and instructive	23	Approved; viewer row now Stretch
D-22	Scenarios are Java classes registered by name; stored as name, parameters, seed, fault plan	JSON or YAML scenario files	Applications are Java anyway; no second configuration language	23	Approved
D-23	OMISSION edge kind: a potential edge, carrying clocks, from a fault event to a TIMER_FIRED event when the fault was active during the timer's wait and touches the node. Phase 12a covers partition membership; crash and pause scopes arrive in 12b	No edge for silence; inferring silence only in the analyser	A dropped message never reaches the watchdog, so without it the graph has no path from partition to promotion and AD-1 and AD-5 lose their explanation	10, 12, 18	New in v0.2
D-24	Events on `env` are keyed by what they concern: `drop:<messageId>`, `fault:<ruleId>#<n>`, `iv:<interventionId>`, `inv:<name>#<n>`. The `env` counter feeds clocks only, never identity	One global `env` counter	One extra `env` event in a branch would renumber every later one, creating false differences and breaking the locality law	10, 19, 20	New in v0.2
D-25	An INTERVENTION_APPLIED event on `env` marks each intervention at the branch point (id `iv:<id>`). `origin` is part of the content digest. A branch with no interventions has no such event, so law M7 still holds	Metadata only, with `origin` ignored by the digest	Gives every intervention a uniform causal root and makes law M10 well defined, including for interventions that remove events	10, 17, 19, 20, 25	New in v0.2

ID	Decision	Alternatives considered	Why this one	Section	Status
D-26	The Node API lives in its own module, `chronos-sdk` (eight modules in total)	Keep the SDK inside runtime with a package rule or a scan	The build itself stops demo applications from reaching engine internals	23	New in v0.2

Open questions

#	Question	Why it matters	Default if unanswered	Status
Q1	What are the deadline and your real weekly hours?	Calibrates Section 24	Milestone S is about 24 joint weeks; default: reach Milestone S first.	**OPEN**
Q2	Maven or Gradle?	Build setup in Phase 3	Maven	Confirmed: Maven
Q3	Is Jackson acceptable as the single runtime library?	D-09	Yes	Confirmed: yes
Q4	Is RE (replicated store with failover) accepted as the flagship demonstration?	Drives Phases 3 to 15	Yes	Confirmed: yes
Q5	Is a real-network live mode permanently out of scope?	D-03	Out of scope	Resolved by D-03: out of v1; a live-network recorder is a post-v1 idea and never enters the replay core

#	Question	Why it matters	Default if unanswered	Status
Q6	What form of evaluation does your institution expect: report, demonstration, viva, paper?	Shapes Phase 17	Demonstration, report and viva	**OPEN**
Q7	Is a declared version string plus a best-effort class hash enough to identify application code?	Section 15	Yes, with a warning on mismatch	Confirmed: yes, with a warning on mismatch
Q8	Which Git hosting and review workflow will you use?	Team workflow	One repository, feature branches, review by the other person	Resolved: one Git repository, feature branches, cross-review by the other teammate before merging (hosting service not named)
Q9	Are KA-2 (lost update) and KA-3 (blocked commit) accepted as the two further known-answer scenarios?	Section 25	Yes	Confirmed: yes
Q10	Will any demo need node states larger than about 10 KB?	D-10 storage policy	No	Confirmed: node states stay under about 10 KB

Remaining open items

- **Open questions:** Q1 (deadline and weekly hours) and Q6 (evaluation requirements of your institution). Neither has been answered or assumed. Q6 is unchanged from v0.1; Q1 changed only in the milestone arithmetic of its default.

- **To validate during implementation:** the D-16 anchor rule (Phase 14a, with KA-1 and a hand-built cross-timeline case); the D-23 omission scope (partitions in Phase 12a, crash and pause in 12b); the cost of per-field last-writer tracking (measure in Phases 8 and 16).